

WebSphere Liberty on z/OS

Basic Configuration



\

Lab Version Date: July 15, 2026

Table of Contents

General Exercise Information and Guidelines	3
Setup and Service Definitions	4
Run the jobs to setup RACF framework for server runtime.....	4
Create an IBM z/OS Connect server.....	10
Summary	13
Customize the JCL start procedures and start the server.....	14
Verify the z/OS Connect server configuration.....	24
Deploying Services and APIs	29
Update the z/OS Connect Server Configuration	29
Deploy the Services	31
Using Postman	31
Using cURL.....	34
Test the Services.....	37
Using Postman	37
Using cURL.....	41
Deploy and Test the API.....	45
Optional	49

Important: There is a folder on the Windows desktop named *CopyPaste Files*. This folder contains file with the commands and other text used in this workshop. Locate the file identified in the *General Exercise Information and Guidelines* section of this exercise and copy it to the desktop. Open the file and use the copy-and-paste function (**Ctrl-C** and **Ctrl-V**) to enter commands or text. It will save time and help avoid typo errors. As a reminder text that appears in this file will be highlighted in yellow.

General Exercise Information and Guidelines

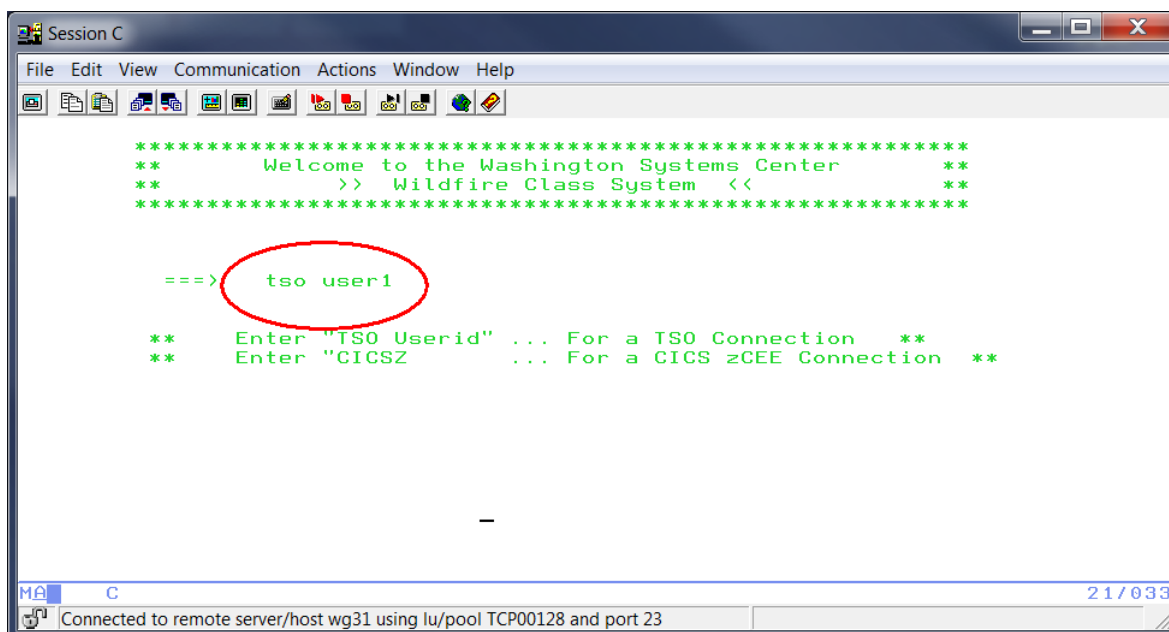
- ✓ This exercise creates and performs the initial configuration of a z/OS Connect Liberty server. Using z/OS Connect provides a good demonstrating of the various configuration options available.
- ✓ This exercise requires using z/OS user identity *USER1*. The RACF password for this user is *user1* (lower case sensitive).
- ✓ Any time you have any questions about the use of IBM z/OS Explorer, 3270 screens, features, or tools, do not hesitate to ask the instructor for assistance.
- ✓ Text in **bold** and highlighted in **yellow** in this document should be available for copying and pasting in a file named *Basic Configuration CopyPaste* file on the desktop.
- ✓ Please note that there may be minor differences between the screen shots in this exercise versus what you see when performing this exercise. These differences should not impact the completion of this exercise.
- ✓ For information regarding the use of the Personal Communication 3270 emulator, see the *Personal Communications Tips* PDF in the exercise folder.

Setup and Service Definitions

Run the jobs to setup RACF framework for server runtime

1. Open the *WG31* icon on the workstation desktop. This will start a 3270-terminal session to your z/OS system.

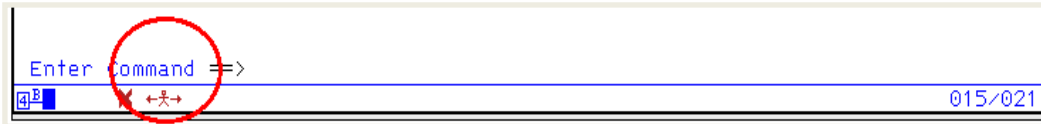
Tech-Tip: Desktop tools can be opened either by double clicking the icon or by selecting the icon and right mouse button clicking and then selecting the *Open* option.



N.B., The 3270-terminal sessions and OMVS screen shots in the remainder of this and subsequent exercises are shown in reverse video simply for printing purposes

2. Enter ***TSO USER1*** (see below) and press the 3270 **Enter** key (the **right-Ctrl** key sequence):
The 3270-emulator used for this workshop (IBM Personal Communication) maps the 3270 enter key to the right **Ctrl** key (see below). Any references to the *Enter* key in non-3270 windows, OMVS terminal session, etc. refers to the key labeled *Enter* on the keyboard.



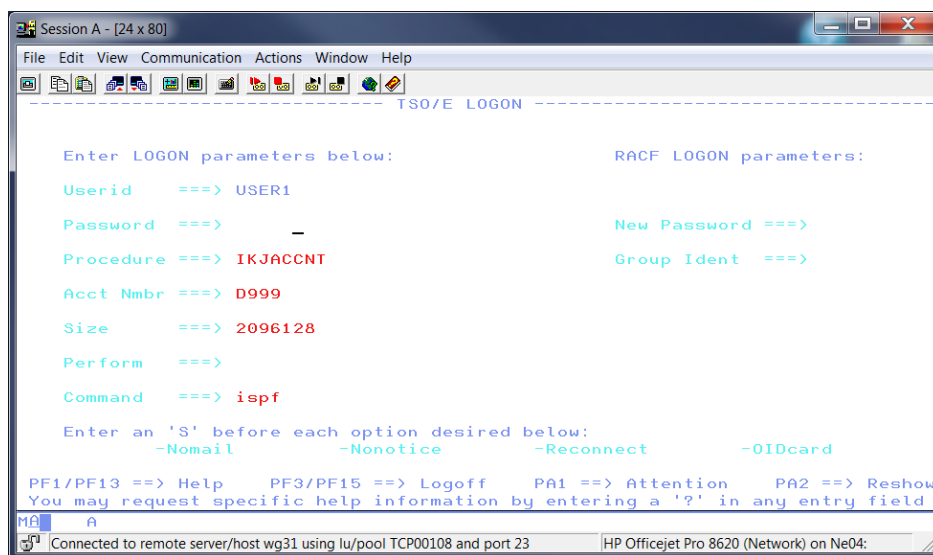


The instructions in this exercise will reference keyboard keys using a **bold** font. In the beginning explicit instructions regarding pressing keyboard keys will be provided. Eventually these explicit instructions to press the *Enter* key for example will not be included. If you are told to enter command, then assume the appropriate *Enter* key should be pressed to have the command invoked or executed. Information which must be entered on a screen or panel will be in ***bold italics***. References to text on a screen or panel will be in normal *italics*.

Tech-Tip: Different 3270-terminal emulators will display an icon like the *Personal Communications*®'s icon below at the bottom of the screen when the keyboard is locked. If this occurs use the left **Ctrl** key to reset or free the keyboard.

In this emulator the **Pause** key is mapped to the clear function. If your laptop has a **Pause** key use it to clear the screen. For newer laptops without a **Pause** key, use the key sequence **Fn-P** to clear the screen. If none of these works, try a **Break** key or an **Alt-C** key sequence. For Apple devices, see the *Personal Communication Tips* document.

3. On the *TSO/E LOGON* panel, enter the password supplied by the instructors.

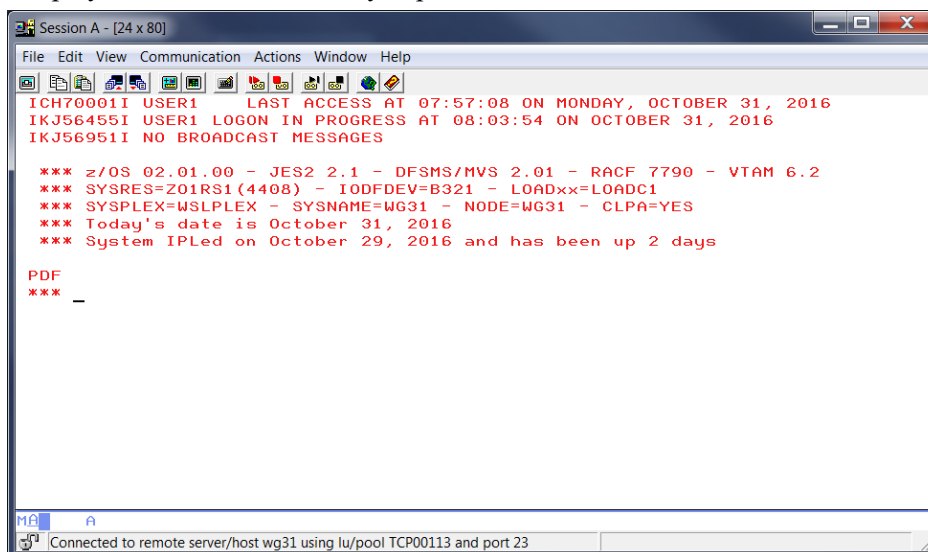


Tech-Tip: An **ISPF** or **PDF** command at the command prompt (==>) on this screen will automatically start ISPF. The copyright information displayed in Step 5 can be bypassed by entering ***ISPF NOLOGO*** or ***PDF NOLOGO*** at the command prompt.

Tech-Tip: If for some reason you are disconnected from your TSO session and cannot log in because your TSO session is still active, you can enter an **S** beside **-Reconnect** near the bottom of the panel to reconnect to your session.

WebSphere Liberty on z/OS - Basic Configuration

4. In a TSO session whenever you see the string ***** (three asterisk)** appear (as below) as the last line in any terminal output, there is more output is waiting to be displayed. Press the **Enter** key when you are ready to see this additional output. Also remember that notification messages such as jobs completing, etc. will not be displayed unless the **Enter** key is pressed

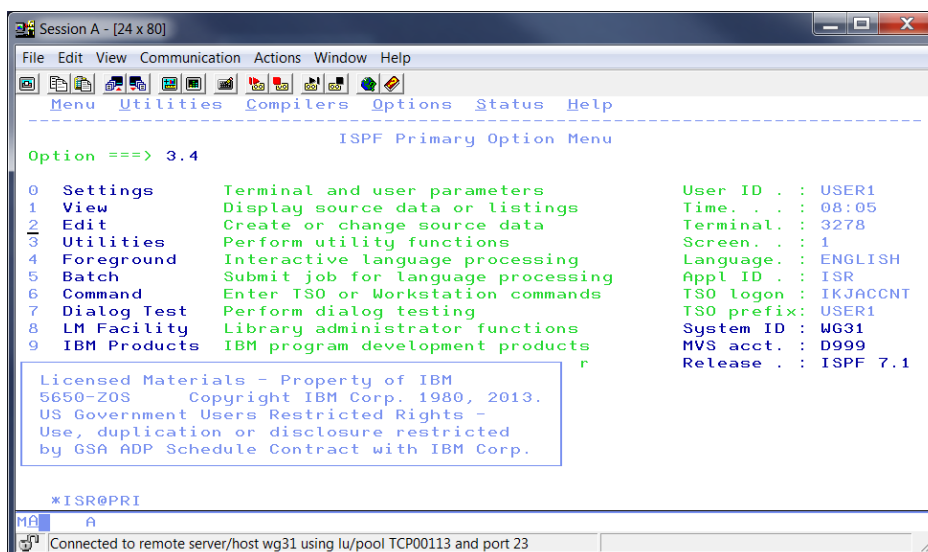


```
Session A - [24 x 80]
File Edit View Communication Actions Window Help
ICH70001I USER1 LAST ACCESS AT 07:57:08 ON MONDAY, OCTOBER 31, 2016
IKJ56455I USER1 LOGON IN PROGRESS AT 08:03:54 ON OCTOBER 31, 2016
IKJ56951I NO BROADCAST MESSAGES

*** z/OS 02.01.00 - JES2 2.1 - DFSMS/MVS 2.01 - RACF 7790 - VTAM 6.2
*** SYSRES=Z01RS1(4408) - IODFDEV=B321 - LOADxx=LOADC1
*** SYSPLEX=WSPLEX - SYSNAME=WG31 - NODE=WG31 - CLPA=YES
*** Today's date is October 31, 2016
*** System IPLed on October 29, 2016 and has been up 2 days

PDF
***
MA A
Connected to remote server/host wg31 using lu/pool TCP00113 and port 23
```

5. You should now be at the main ISPF panel (see below). Press the **Enter** key to dismiss the *Copyright* statement.



```
Session A - [24 x 80]
File Edit View Communication Actions Window Help
Menu Utilities Compilers Options Status Help
-----
ISPF Primary Option Menu
Option ==> 3.4

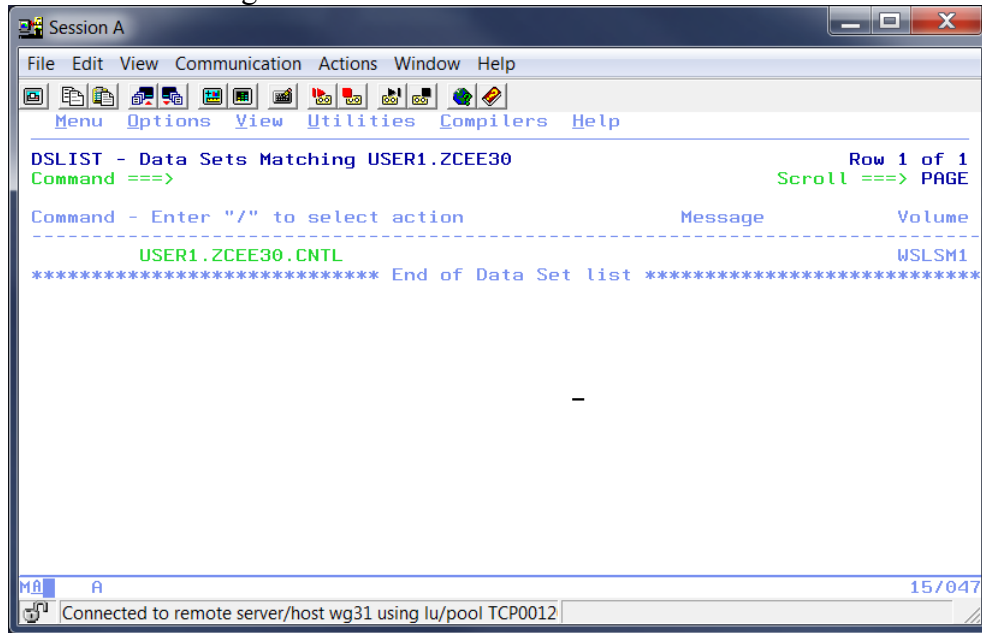
0 Settings      Terminal and user parameters      User ID . : USER1
1 View         Display source data or listings    Time . . : 08:05
2 Edit         Create or change source data       Terminal. : 3278
3 Utilities    Perform utility functions         Screen . : 1
4 Foreground   Interactive language processing   Language. : ENGLISH
5 Batch        Submit job for language processing  Appl ID . : ISR
6 Command      Enter TSO or Workstation commands  TSO logon : IKJACCNT
7 Dialog Test  Perform dialog testing            TSO prefix: USER1
8 LM Facility  Library administrator functions   System ID : WG31
9 IBM Products IBM program development products MVS acct. : D999
                                           Release . : ISPF 7.1

Licensed Materials - Property of IBM
5650-Z0S Copyright IBM Corp. 1980, 2013.
US Government Users Restricted Rights -
Use, duplication or disclosure restricted
by GSA ADP Schedule Contract with IBM Corp.

*ISR@PRI
MA A
Connected to remote server/host wg31 using lu/pool TCP00113 and port 23
```

6. Enter ISPF command **3.4** in the area after the *Option==>* prompt in the upper left (see above) and press the **Enter** key to display the *Data Set List Utility* panel.

7. On the *Data Set List Utility* panel, enter **USER1.ZCEE30** in the area beside *DSNAME level*, and press the **Enter** key to display a list of data sets whose names begin with *USER1.ZCEE30*. You should see something like the list below:



8. Enter **E** (for "edit") next to the data set *USER1.ZCEE30.CNTL*, and press **Enter**. You should see a list of members in the partitioned data set.

9. Enter **B** (for "browse") next to member **ZCEERSTC**, and press **Enter**: You should see a job with several RACF commands (see below):

```
ADDGROUP LIBGRP OMVS(AUTOGID) OWNER(SYS1)

ADDUSER LIBANGE DFLTGRP(LIBGRP) OMVS(AUTOUID HOME(/u/libange/) -
  PROGRAM(/bin/sh)) NAME('LIBERTY ANGEL') NOPASSWORD NOOIDCARD
ADDUSER LIBSERV DFLTGRP(LIBGRP) OMVS(AUTOUID HOME(/u/libserv/) -
  PROGRAM(/bin/sh)) NAME('LIBERTY SERVER')
ALTUSER LIBSERV PASSWORD(LIBSERV) NOEXPIRED

RDEFINE STARTED BAQSTRT.* UACC(NONE) -
  STDATA(USER(LIBSERV) GROUP(LIBGRP) -
  PRIVILEGED(NO) TRUSTED(NO) TRACE(YES))
RDEFINE STARTED BAQZANGL.* UACC(NONE) -
  STDATA(USER(LIBANGE) GROUP(LIBGRP) -
  PRIVILEGED(NO) TRUSTED(NO) TRACE(YES))
SETROPTS RACLIST(STARTED) REFRESH

RDEFINE SURROGAT BPX.SRV.LIBSERV
RDEFINE SURROGAT LIBSERV.SUBMIT
PERMIT BPX.SRV.LIBSERV CLASS(SURROGAT) ID(ZCEEADM) ACC(READ)
PERMIT LIBSERV.SUBMIT CLASS(SURROGAT) ID(ZCEEADM) ACC(READ)
SETROPTS RACLIST(SURROGAT) REFRESH
```

Tech-Tip: This job creates a RACF group and IDs for the Angel process and an IBM z/OS Connect servers. It then it creates two STARTED task profiles with these IDs assigned to these started tasks. Finally, the RACF resources that allows members of a z/OS Connect administrators group (ZCEEADM) to act as a surrogate of user LIBSERV and/or submit jobs as LIBSERV are defined.

10. Enter command **SUBMIT** in the area after the command prompt (*Command ==>*) at the top of the screen and press the **Enter** key to submit this job for execution. You should get a message indicating the job has been submitted, along with the three asterisks indicating that additional output is being held before being displayed:

```
IKJ56250I JOB ZCEERSTC(JOB00060) SUBMITTED
***
```

11. Press the **Enter** to display the additional output. You should then see either a message indicating the job has completed with MAXCC=0000 (which is good) or a redisplay of the ISPF browse panel. If the latter, keep pressing **Enter** until the job completes.

```
08.19.07 JOB00060 $HASP165 ZCEERSTC ENDED AT WG31 MAXCC=0000 CN(INTERNAL)
***
```

12. Press the **F3** key to return to the list of members.

13. Enter **B** next to the **ZCEERSVR** member and press the **Enter** key. This is another set of RACF commands that define the RACF *SERVER* resources which allow the use of various z/OS authorized services. *WP102604 Getting Started Guide* describes these commands in more detail. **Submit** this job for execution (see Step 10 above). Allow this job to complete before continuing.

```
RDEFINE SERVER BBG.ANGEL UACC(NONE) OWNER(SYS1)
PERMIT BBG.ANGEL CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSAFM UACC(NONE) OWNER(SYS1)
PERMIT BBG.AUTHMOD.BBGZSAFM -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSAFM.SAFCRED UACC(NONE)
PERMIT BBG.AUTHMOD.BBGZSAFM.SAFCRED -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSAFM.ZOSWLM UACC(NONE)
PERMIT BBG.AUTHMOD.BBGZSAFM.ZOSWLM -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSAFM.TXRRS UACC(NONE)
PERMIT BBG.AUTHMOD.BBGZSAFM.TXRRS -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSAFM.ZOSDUMP UACC(NONE)
PERMIT BBG.AUTHMOD.BBGZSAFM.ZOSDUMP -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.SECPF.BBGZDFLT UACC(NONE)
PERMIT BBG.SECPF.BBGZDFLT -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSAFM.WOLA UACC(NONE) OWNER(SYS1)
PERMIT BBG.AUTHMOD.BBGZSAFM.WOLA -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSAFM.LOCALCOM UACC(NONE) OWNER(SYS1)
PERMIT BBG.AUTHMOD.BBGZSAFM.LOCALCOM -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSCFM UACC(NONE) OWNER(SYS1)
PERMIT BBG.AUTHMOD.BBGZSCFM -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSCFM.WOLA UACC(NONE) OWNER(SYS1)
PERMIT BBG.AUTHMOD.BBGZSCFM.WOLA -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
PERMIT BBG.AUTHMOD.BBGZSCFM.WOLA -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSAFM.PRODMGR UACC(NONE) OWNER(SYS1)
PERMIT BBG.AUTHMOD.BBGZSAFM.PRODMGR -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
RDEFINE SERVER BBG.AUTHMOD.BBGZSAFM.ZOSAIO UACC(NONE) OWNER(SYS1)
PERMIT BBG.AUTHMOD.BBGZSAFM.ZOSAIO -
  CLASS(SERVER) ACCESS(READ) ID(LIBSERV)
SETROPTS RACLIST(SERVER) REFRESH
```

14. Press **F3** to exit the browse session.

Summary

You just created a set of essential SAF profiles for z/OS Connect EE V3.0 to use. These are detailed in the *WP102604 Getting Started Guide*. The process was streamlined for this lab by coding them in a job so submitting this job would create what was needed.

Create an IBM z/OS Connect server

IBM z/OS Connect V3.0 was installed using SMP/E ahead of the workshop and the *zconsetup* command has been invoked to create a symbolic link to the */var/zosconnect/v3r0/extensions* directory. This directory contains property files which provide information required for locating IBM z/OS Connect features and executables by Liberty at runtime.

Now it's time to create a z/OS Connect server. This is done by executing a shell script (*zosconnect*) provided with IBM z/OS Connect.

Tech-Tip: The OMVS directories and files created by this script must be owned by the same RACF identity associated with the z/OS Connect started task (e.g. BAQSTR). This is problematic since this RACF identity is restricted and cannot be used to logon to TSO or submit jobs to create these directories and files.

There are various ways to ensure this ownership is set correctly. One is to create the directories and files invoking the OMVS commands using your regular RACF identity. Then use the Unix System Services (USS) command *chown* (change owner) to change the ownership of the directories and files to the RACF identity of the started task.

1. Enter **B** next to the **MYSERVER** member and press the **Enter** key. This job exports a set of environment variables and starts an OMVS shell. The exported environment variables are used in a series of OMVS commands. The first two commands export the `JAVA_HOME` and `WLP_USER_DIR` environment variables so these variables will be available to processes spawned in the shell. Next the `zosconnect` command is invoked to create the server's directory structure and initial configuration files. Next a symbolic link is created from the shared configuration directory to a directory shared by multiple servers on this image. Finally, the ownership of the server's configuration directory structure is set of the SAF identity under which the server will execute. **Submit** this job for execution (see Step 10 above). Allow this job to complete before continuing.

```
//MYSERVER JOB 'ZCEE',CLASS=A,REGION=0M,MSGCLASS=H,NOTIFY=&SYSUID
//*****
//* SET SYMBOLS
//*****
//EXPORT EXPORT SYMLIST=(*)
// SET JAVAHOME='/usr/lpp/java/J8.0_64'
// SET ZCEEPATH='/usr/lpp/IBM/zosconnect/v3r0'
// SET SERVER='myServer'
// SET TEMPLATE='zosconnect:default'
// SET WLPUSER='/var/zosconnect'
// SET USER='LIBSERV'
// SET GROUP='LIBGRP'
//*****
//* Step ZCEESRVR - Use the zosconnect command to create a server
//*****
//ZCEESRVR EXEC PGM=IKJEFT01,REGION=0M
//SYSTSPRT DD SYSOUT=*
//SYSERR DD SYSOUT=*
//STDOUT DD SYSOUT=*
//SYSTSIN DD *,SYMBOLS=EXECSYS
BPXBATCH SH +
export JAVA_HOME=&JAVAHOME; +
export WLP_USER_DIR=&WLPUSER; +
&ZCEEPATH/bin/zosconnect create &SERVER +
--template=&TEMPLATE; +
ln -s $WLP_USER_DIR/shared/config +
/var/zosconnect/servers/myServer/includes; +
chown -R &USER:&GROUP $WLP_USER_DIR/servers/&SERVER
```

Tech-Tip: Other templates which can be specified with the `zosconnect create` command are `zosconnect:apiRequester`, `zosconnect:sampleCicsIpicCatalogManager`, `sampleDb2ProjectManager`, `zosconnect:sampleImsDatabase`, `zosconnect:sampleImsPhonebook`, `sampleMqStockManager` and `zosconnect:sampleWolaCatalogManager`,

The best template to use to create a server is `zosconnect:apiRequester`. This template creates all resource sub directories. If the default template is used the same sub directories are not created.

___ 2. In the existing 3270-terminal session, enter TSO command **OMVS** (e.g. **TSO OMVS**) at the command prompt and press **Enter** to start an ISPF OMVS shell session.

___ 3. Enter the following OMVS command to display the contents of the `/var/zosconnect/servers` subdirectory:

```
ls -al /var/zosconnect/servers
```

You should see something like this:

```
$ ls -al /var/zosconnect/servers
total 96
drwxr-x--T   6 LIBSERV  SYS1      8192 Oct 31 11:57 .
drwxrwxrwx   4 BAGWELL  SYS1      8192 Oct 29 15:17 ..
drwxr-x--T   3 LIBSERV  SYS1      8192 Oct 29 15:17 .classCache
drwxr-x---   2 LIBSERV  SYS1      8192 Oct 31 11:57 .logs
drwxr-x---   3 LIBSERV  SYS1      8192 Oct 31 11:57 myServer
```

The *myServer* directory was created by *zosconnect* command in the previous step.

___ 4. Now display the contents of the `/var/zosconnect/servers/myServer` directory by entering OMVS command:

```
ls -al /var/zosconnect/servers/myServer
```

You should see:

```
$ ls -al /var/zosconnect/servers/myServer
total 66
drwxr-x---   3 LIBSERV  SYS1      8192 Oct 31 11:57 .
drwxr-x--T   6 LIBSERV  SYS1      8192 Oct 31 11:57 ..
lrwxrwxrwx   1 USER1    OMVSGRP    39 Oct 31 11:57 includes -> /var/zosconnect/shared/config
drwxr-x---   4 LIBSERV  SYS1      8192 Aug  1 16:23 resources
-rw-r-----   1 LIBSERV  SYS1       25 Oct 31 11:57 server.env
-rw-r-----   1 LIBSERV  SYS1      442 Oct 31 11:57 server.xml
drwxr-x---   3 LIBSERV  SYS1      8192 Oct 31 11:57 workarea
```

The "server.xml" file is the key configuration file for the Liberty z/OS server.

A symbolic link was previously created from the Liberty server's shared configuration directory to the LPAR's common configuration directory `/var/liberty/includes`.

This ensure that the directory referenced by the *shared.config.dir* environment variable references the common configuration directory shared by all the Liberty servers on this LPAR

Tech-Tip: Symbolic links have been created from a single directory to each of the server configuration directories on this image to simplify administration. Directory `/var/liberty` was created and the command `ln -s /var/zosconnect/servers/myServer /var/liberty/myServer` was entered at an OMVS prompt to create this symbolic linkage for this server.

Start an OMVS shell and enter command `ls -al /var/liberty` at the command prompt. You should see something like what is shown below:

```
lrwxrwxrwx  1 USER1      SYS1              32 Jan 31 12:43 myServer ->
/var/zosconnect/servers/myServer
lrwxrwxrwx  1 JOHNSON    SYS1              36 Sep 10 08:40 wlpoidop ->
/var/ats/zosconnect/servers/wlpoidop
lrwxrwxrwx  1 JOHNSON    SYS1              36 Sep 10 08:39 wlpoidrp ->
/var/ats/zosconnect/servers/wlpoidrp
lrwxrwxrwx  1 JOHNSON    SYS1              14 Jan 15 2019 zc3lab -> /wasetc/zc3lab
lrwxrwxrwx  1 JOHNSON    SYS1              36 Jan 15 2019 zceeapir ->
/var/ats/zosconnect/servers/zceeapir
lrwxrwxrwx  1 JOHNSON    SYS1              39 Jan 15 2019 zceecics ->
var/cicsts/zosconnect/servers/zceecics
lrwxrwxrwx  1 JOHNSON    SYS1              25 Jan 15 2019 zceehats ->
/var/wlp/servers/zceehats
lrwxrwxrwx  1 JOHNSON    SYS1              36 Sep 12 11:14 zceeopid ->
/var/ats/zosconnect/servers/zceeopid
lrwxrwxrwx  1 JOHNSON    SYS1              36 Jan 15 2019 zceesrv1 ->
/var/ats/zosconnect/servers/zceesrv1
lrwxrwxrwx  1 JOHNSON    SYS1              36 Jan 15 2019 zceesrv2 ->
/var/ats/zosconnect/servers/zceesrv2
lrwxrwxrwx  1 JOHNSON    SYS1              36 Jul  2 2019 zceesrv3 ->
/var/ats/zosconnect/servers/zceesrv3
```

Notice that the use of symbolic links provides a single directory for accessing the different Liberty servers configured on this LPAR. These servers were configured with 4 different values for the WLP_USER_DIR environment variable, `/var/zosconnect`, `/var/ats/zosconnect`, `/var/wlp` and `/var/cicsts/zosconnect`. There is also a symbolic link to a common or shared directory `/var/zee/includes` was also created for containing common configuration elements.

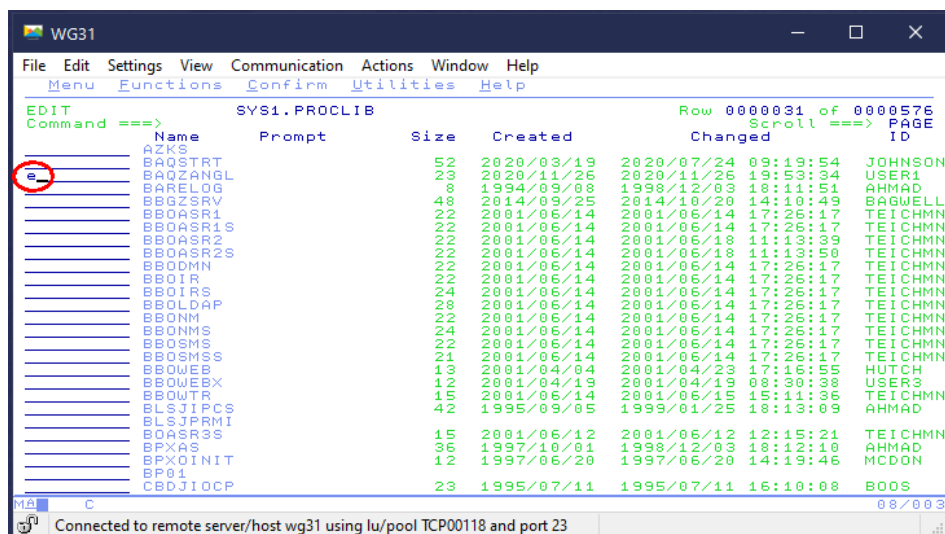
5. Terminate the OMVS session by entering the **exit** command twice to redisplay an ISPF panel. The first exit terminates the *libserv* session, and the second exit terminates the *user1* session.

Summary

You just created a server, but right now it's just a skeleton without any service definitions or deployed APIs. Next you will customize the JCL start procedures for the Angel process and the server.

Customize the JCL start procedures and start the server

- ___ 1. In your 3270-terminal session, enter ISPF command =3.4 in the command prompt area and press **Enter**.
- ___ 2. Enter **SYS1.PROCLIB** in the area beside *Dsname Level* and press **Enter**.
- ___ 3. Type an **E** (edit) next to the **SYS1.PROCLIB** data set and press **Enter**.
- ___ 4. You will now see a long list of members in the **SYS1.PROCLIB** data set. Enter command **L BAQ** after the command prompt (Command ==>) and press **Enter**. Locate the **BAQZANGL** member and place an **E** (edit) next to **BAQZANGL** to open this member in edit mode simply to view this member.



This JCL procedure was copied this member to **SYS1.PROCLIB** prior to the workshop. This JCL is supplied in the z/OS Connect EE V3.0 SBAQSAMP target data set.

N.B. Instructions to press the **Enter** key should be assumed in subsequent steps as appropriate to execute commands, etc.

- ___ 5. Press **F3** to exit the edit session.
- ___ 6. Scroll up (**F7**) and locate member **BAQSTRT**. Place an **E** next to the member in order to open the member so changes can be made to the member.

This is the JCL procedure used to start the z/OS Connect server. This JCL was copied to **SYS1.PROCLIB** prior to the workshop as well.

You are going to make three changes to this JCL procedure. The first is to replace the first line so the **PARMS=** specifies your server's name¹.

¹ You could start the server with **S BAQSTRT,PARMS='myServer'** and avoid updating this line in the JCL. We chose to hard code the server's name for this workshop so the start is a simple **S BAQSTRT**.

___ 7. Clear line 1 by placing your cursor at the start of the line (on the first slash in column 1):

```
000001 //BAQSTRT  PROC PARMS='defaultServer'  
000002 /**  
000003 /** Licensed Materials - Property of IBM
```

___ 8. Then press the **End** key. That should clear the line of all characters.

___ 9. Copy/Paste the JCL statement below on line 1 by using **Ctrl-C** of the text from the copy/paste file and then selecting *Edit* → *Paste* or **Ctrl-V** in the 3270-terminal session.

```
//BAQSTRT  PROC PARMS='myServer'
```

The result should be:

```
000001 //BAQSTRT  PROC PARMS='myServer'  
000002 /**  
000003 /** Licensed Materials - Property of IBM
```

Important: In the above instructions the *Edit* → *Paste* translates to click the *Edit* tool in the tool bar of the 3270-terminal session and then select the *Paste* option. The → arrow will be used in later steps for selecting other tools in a tool bar and selecting an option in the 3270-terminal session and other windows.

You could have just overtyped *defaultServer* with *myServer*. We chose to go with copy-and-paste to eliminate any chance of a case mismatch or typo. The field was cleared first because the original text in the line was longer than its replacement.

___ 10. The next two lines to be updated are the `// SET ZCONHOME=` (line 38) and the `JAVA_HOME=` (line 47) environment variables. Do the following:

- Scroll down (**F8**) until you see the `SET ZCONHOME=` line:
- Place your cursor at the start of the line (the first slash):
- From the copy-and-paste file, copy the line:

```
// SET ZCONHOME='/usr/lpp/IBM/zosconnect/v3r0'
```

```
000037 // *
000038 // SET ZCONHOME='/usr/lpp/IBM/zosconnect/v3r0'
000039 // *
```

- Scroll down (**F8**) and locate the `JAVA_HOME=` line. It should be line 47:
- Place your cursor at the start of that line and press the **End** key to clear the line.
- From the copy-and-paste file, copy the line:

```
JAVA_HOME=/usr/lpp/java/J8.0_64
```

- Place your cursor at the start of that line and press the **End** key to clear the line.
- Locate the `WLP_USER_DIR=` line. It should be line 48:
- Place your cursor at the start of that line and press the **End** key to clear the line.
- From the copy-and-paste file, copy the line:

```
WLP_USER_DIR=/var/zosconnect
```

- In your 3270-terminal session, select *Edit* → *Paste* (you may need to remove a trailing > character). The result should be:

```
000045 //STDENV DD *
000046 _BPX_SHAREAS=YES
000047 JAVA_HOME=/usr/lpp/java/J8.0_64
000048 WLP_USER_DIR=/var/zosconnect
000049 #JVM_OPTIONS=<Optional JVM parameters>
```

Now you're ready to start the *Angel* process and the server.

Tech-Tip: An additional DD statement can be added to the JCL which will provide useful information regarding the status of the z/OS Connect. Normally detailed Liberty messages are written to an OMVS file in ASCII but if you add a DD statement for DD name MSGLOG and specify SYSOUT=* a subset of these messages will be included in the SPOOL of the task.

___ 11. Go to SDSF in your 3270-terminal session by entering ISPF command `=sdsf.da` in the command field and pressing **Enter**. The *da* stands for *display active*, and it is the z/OS facility for show running tasks and completed jobs.

___ 12. Next, enter **PRE BAQ*** in the command field and press **Enter**. This sets a "prefix" so only tasks starting with string *BAQ* are shown. The Angel task starts with *BAQ*. After entering that command, you should see the new prefix is in effect.

13. Enter the MVS command ***S BAQZANGL*** after the command prompt.

Tech-Tip: MVS and JES2 commands can be entered from SDSF by enter a / (slash) on the command line followed by the command itself (e.g. /D T). The command results can be found in the system log. If a command is especially long, then enter a / (slash) to display a *SDSF – System Command Extension* panel where a command can span multiple lines. When a MVS command must be entered, the instructions in these exercises will indicate that the command is a MVS command and you may enter the command at the prompt by using the / (slash) prefix or using the *SDSF – System Command Extension* panel.

14. Keep pressing the **Enter** key until you should see the Angel process is running.

Display Filter View Print Options Search Help											

SDSF DA	WG31	WG31	PAG	0	CPU	0	LINE 1-1 (1)				
COMMAND INPUT ==> SCROLL ==> CSR											
NP	JOBNAME	StepName	ProcStep	JobID	Owner	C	Pos	DP	Real	Paging	SIO
	BAQZANGL	BAQZANGL	STEP1	STC00045	LIBANGE	NS	FE	352	0.00	0.00	

15. You should see *LIBANGE* is the ID that owns the started task. That is a result of the RACF generated by the *ZCEESTC* job you ran earlier. The STARTED profile matched to the *BAQZANGL* start procedure and assigned the ID *LIBANGE* to this task.

Display Filter View Print Options Search Help											

SDSF DA	WG31	WG31	PAG	0	CPU	0	LINE 1-1 (1)				
COMMAND INPUT ==> SCROLL ==> CSR											
PREFIX=BAQ* DEST=(ALL) OWNER=* SYSNAME=											
NP	JOBNAME	CPU-Time	ProcStep	SR	DP	Pos	C	Owner	Status	SysName	SPag SCP
	BAQZANGL	0.00	STEP1	FE	NS			LIBANGE		WG31	0

16. Enter the MVS command ***S BAQSTRT*** after the command prompt and press **Enter** until you see an active BAQSTRT task.

Display Filter View Print Options Search Help											

SDSF DA	WG31	WG31	PAG	2	CPU	8	LINE 1-1 (1)				
COMMAND INPUT ==> SCROLL ==> CSR =											
NP	JOBNAME	StepName	ProcStep	JobID	Owner	C	Pos	DP	Real	Paging	SIO
	BAQSTRT	BAQSTRT	ZCON	STC00048	LIBSERV	IN	F8	10T	0.00	0.00	
	BAQZANGL	BAQZANGL	STEP1	STC00045	LIBANGE	LO	FF	395	0.00	0.00	

You should see that *LIBSERV* is the ID that owns the started task. That is a result of the RACF generated by the *ZCEESTC* job you ran earlier. The BAQSTRT started procedure matched STARTED profile resource and the task was assigned the ID *LIBSERV* to it.

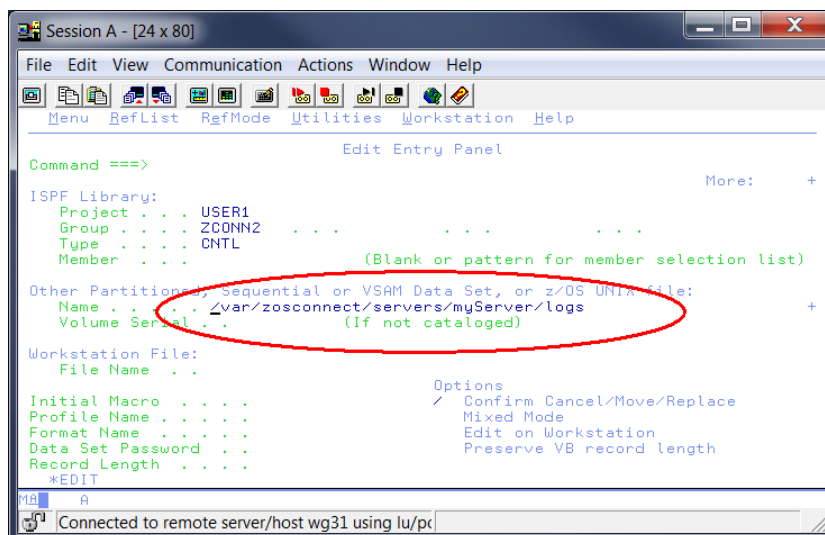
Note: The z/OS Connect server is running with *LIBSERV*'s RACF authority. This is why it was important to run the *zosconnect create* command as *LIBSERV* and why it is important in subsequent steps to use the *LIBSERV* identity when deploying Service ARhive (SAR) files. Also note that in later steps when files are included into the *server.xml* configuration file, the files that are included are owned by *LIBSERV*. All of this is done to avoid OMVS permission issues.

Both the Angel process and the z/OS Connect V3.0 server are started. Let's go look at the *messages.log* file for the server.

20. Go to the *ISPF Edit Entry Panel* (option 2) by entering **=2** on the command line and pressing **Enter**.

Tech-Tip: Most of the subsequent steps in this section can be performed using the IBM z/OS Explorer. If you are interested in doing these steps using this Eclipse tool, contact the instructor.

21. Enter ***/var/zosconnect/servers/myServer/logs*** into the area beside *Name* under *Other Partitioned, Sequential or VSAM Data Set, or z/OS UNIX file:* and press **Enter**.



Tech-Tip: Try using the symbolic link discussed earlier, e.g. */var/zcee/myServer/logs* instead.

22. This will display the contents of the OMVS directory where the z/OS Connection messages are written to file *messages.log*.

```
-----
                        z/OS UNIX Directory List                      Row 1 to 4 of 4
Command ===>                                           Scroll ===> PAGE
Time zone EST5EDT is used to calculate the displayed date and time values.
Pathname . : /SYSTEM/var/zosconnect/servers/myServer/logs
EUID . . . : 8470391
Command  Filename                                Message                Type Permission
-----
      .                                           Dir   rwxrwxrwx
      ..                                          Dir   rwxr-x---
      messages.log                               File  rw-rw-rw-
      state                                       Dir   rwxrwxrwx
***** Bottom of data *****
```

23. Messages are written in ASCII so to view them in a 3270-terminal session use the *VA* (View ASCII) line command. Enter line command *VA* beside *messages.log* and press **Enter** twice.

24. You should see the *View* panel open and the *messages.log* file contents displayed in EBCDIC:

[illegible]

25. You need to scroll to the right to see more message details. Enter **100** after the command prompt (==>) and press the **F11** key.

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
-----
VIEW          /SYSTEM/var/zosconnect/servers/myServer/logs/me Columns 00101 00172
Command ==>                                         Scroll ==> CSR
***** Top of Data *****
0.22.c1180320180905-2337)

WKE0001I: The server myServer has been launched.
WKB0103I: Authorized service group KERNEL is available.
WKB0103I: Authorized service group LOCALCOM is available.
WKB0103I: Authorized service group PRODMGR is available.
WKB0103I: Authorized service group SAFCRED is available.
WKB0103I: Authorized service group TXRRS is available.
WKB0103I: Authorized service group WOLA is available.
WKB0103I: Authorized service group ZOSAIO is available.
WKB0103I: Authorized service group ZOSDUMP is available. .

```

Note that the **F8** key can be used to scroll forward and the **F7** key can be used to scroll backward. **F10** can be used to scroll leftward and **F11** key can be used to scroll rightward. A numeric value on the command prompt will be used to determine the number of lines to scroll forward or backward or the number of columns for scrolling leftward or rightward.

26. Look at the messages and note the following:

```
000012 WKB0103I: Authorized service group KERNEL is available.
000013 WKB0103I: Authorized service group LOCALCOM is available.
000014 WKB0103I: Authorized service group PRODMGR is available.
000015 WKB0103I: Authorized service group SAFCREd is available.
000016 WKB0103I: Authorized service group TXRRS is available.
000017 WKB0103I: Authorized service group WOLA is available.
000018 WKB0103I: Authorized service group ZOSAIO is available.
000019 WKB0103I: Authorized service group ZOSDUMP is available.
000020 WKB0103I: Authorized service group ZOSWLM is available.
000021 WKB0103I: Authorized service group CLIENT.WOLA is available.
000022 WKB0108I: IBM Corp product z/OS Connect version 03.00 successfully regis
000023 WKB0112I: The number of successfully registered products with z/OS is 1.
000024 WKE0002I: The kernel started after 3.05 seconds
000025 WKF0007I: Feature update started.
000026 WKS0007I: The security service is starting...
000027 NA1001I: WebSphere Dynamic Cache instance named baseCache initialized su
000028 NA1071I: The cache provider default is being used.
000029 NA1056I: Dynamic Cache (object cache) initialized successfully.
000030 WKO0229I: Native Asynchronous I/O support for z/OS has been activated.
000031 WKS4103I: Creating the LTPA keys. This may take a few seconds.
000032 WKS1123I: The collective authentication plugin with class name NullColle
000033 QR0000I: z/OS Connect Enterprise Edition version 3.0.15. (20181120-1404)
000034 WKO0219I: TCP Channel defaultHttpEndpoint has been started and is now li
000035 WKS4104A: LTPA keys created in 1.513 seconds. LTPA key file: /var/zoscon
000036 WKS4105I: LTPA configuration is ready after 1.527 seconds.
000037 WKF0015I: The server has the following interim fixes active in the runti
000038 WKF0012I: The server installed the following features: [servlet-3.1, ssl
000039 WKF0008I: Feature update completed in 5.539 seconds.
000040 WKF0011I: The server myServer is ready to run a smarter planet.
000041 VE0169I: Loading Web Module: z/OS Connect.
000042 VE0250I: Web Module z/OS Connect has been bound to default_host.
000043 WKT0016I: Web application available (default_host): http://wg31.washingt
000044 SN8501I: The session manager did not find a persistent storage location;
000045 SN0176I: A new session context will be created for application key defau
000046 SN0172I: The session manager is using the Java default SecureRandom impl
000047 NA1056I: Dynamic Cache (object cache) initialized successfully.
000048 VE0242I: [com.ibm.zosconnect] [/] [com.ibm.zosconnect.internal.web.Servi
000049 WKS9122I: For URL /* in application com.ibm.zosconnect, the following H
```

1. The "Authorized service is available" messages indicate the SERVER profiles (ZCEESAF job) are in effect for this server and the ID under which it runs. Of particular interest is SAFCREd (for later when we use SAF for security).
2. Version 3.0.15.0 (20181120-1404) indicates the maintenance made available in November 20th of 2018 is in effect. The values you see may be different.
3. The server is started and ready to run.

27. Use the **F3** key to go back to the *Edit Entry Panel* and use the space bar or **Backspace** key to remove the `/logs` subdirectory from the directory name field. Press **Enter** to display the contents of `/var/zosconnect/servers/myServer`. You should see:

```

Menu  Utilities  View  Options  Help
-----
                                z/OS UNIX Directory List                                Row 1 to 8 of 8
Command ===>                                                           Scroll ===> PAGE
Time zone EST5EDT is used to calculate the displayed date and time values.
Pathname . : /SYSTEM/var/zosconnect/servers/myServer
EUID . . . : 8470391
Command  Filename                                          Message                                Type Permission
-----
.                                                Dir  rwxr-x--
..                                               Dir  rwxr-x--T
logs                                             Dir  rwxrwxrwx
resources                                       Dir  rwxrwxrwx
server.env                                     File rw-r-----
server.xml                                     File rw-r-----
workarea                                       Dir  rwxr-x--
***** Bottom of data *****

```

28. Use the **EA** (Edit ASCII) line command to open the `server.xml` file in edit mode. (The command **VA** would open the file in view or browse mode) You should see:

```

000001 <?xml version="1.0" encoding="UTF-8"?>
000002 <server description="new server">
000003
000004     <!-- Enable features -->
000005     <featureManager>
000006         <feature>zosconnect:zosConnect-2.0</feature>
000007         <feature>zosconnect:zosConnectCommands-1.0</feature>
000008     </featureManager>
000009
000010     <!-- To access this server from a remote client add a host attribute
000011     <httpEndpoint id="defaultHttpEndpoint"
000012         host="*"
000013         httpPort="9080"
000014         httpsPort="9443" />
000015
000016     <!-- add cors to allow cross origin access, e.g. when using swagger
000017     <cors id="defaultCORSConfig"
000018         domain="/"
000019         allowedOrigins="*"
000020         allowedMethods="GET, POST, PUT, DELETE, OPTIONS"
000021         allowedHeaders="Origin, Content-Type, Authorization"
000022         allowCredentials="true"
000023         maxAge="3600"/>
000024
000025     <!-- NOTE: Disabling automatic polling for changes to configuration fil
000026     deployed services and APIs is a prudent option for z/OS Connect EE
000027     Polling might be convenient for iterative development and test
000028     systems, but not for production.
000029
000030     Configuration elements that can drive significant polling activity
000031     default are specified below to explicitly disable automatic polling.
000032     Further element types to consider for polling interval include
000033     zosconnect_zosConnectDataXform (default 2 seconds) and
000034     keyStore (default 500 milliseconds).

```

```

000035
000036 Consider setting the updateTrigger attribute to "polled" if changes
000037 to associated resources need to be picked up automatically, and tune
000038 the polling interval accordingly. The attribute that controls polling
000039 frequency for each of these elements is included, together with its
000040 associated default value.
000041 -->
000042
000043 <!-- config requires updateTrigger="mbean" for REFRESH command support
000044 <config updateTrigger="mbean" monitorInterval="500"/>
000045
000046 <!-- zosConnect APIs -->
000047 <zoscconnect_zosConnectAPIs updateTrigger="disabled" pollingRate="5s"
000048
000049 <!-- zosConnect Services -->
000050 <zoscconnect_services updateTrigger="disabled" pollingRate="5s"/>
000051
000052 <!-- applicationMonitor is not applicable for z/OS Connect EE servers -
000053 <applicationMonitor updateTrigger="disabled" dropinsEnabled="false"/>
000054
000055 </server>

```

That is a *very* minimal server.xml file created from the default template. Other templates will generate server.xml files based on their parameters.

In the next step we will have you copy in a new server.xml file that will add quite a few more XML elements. The dynamic nature of Liberty will incorporate the new configuration without requiring a server restart.

Tech-Tip: Note that by default the *updateTrigger* attributes are disabled for APIs and services. These will need to be enabled if auto update is desired.

___29. Add the following include after the *<server description=<new server">* line.

<include location="\$shared.config.dir}/basicSecurity.xml"/>

Tech-Tip: This include element is taking advantage of an environment variable (*shared.config.dir*) whose value is set to the directory containing the *server.xml* file used to initialize this instance of a Liberty server. The *includes* subdirectory is the symbolic link that references directory */var/zcee/includes* (the location of the include files shared among multiple servers. The value of using environment variables and symbolic links in the server.xml makes this configuration element reusable across multiple instances of servers and multiple OMVS directory configurations.

- ___ 30. Including this file (see below) adds a new feature to the server, *appSecurity-2.0*, which enables application security and some of the other elements required for basic security.

```
<server description="basic security">

  <!-- Enable features -->
  <featureManager>
    <feature>appSecurity-2.0</feature>
  </featureManager>

  <keyStore id="defaultKeyStore" password="Liberty"/>

  <webAppSecurity allowFailOverToBasicAuth="true" />

  <basicRegistry id="basic1" realm="zosConnect">
    <user name="Fred" password="fredpwd" />
  </basicRegistry>

  <authorization-roles id="zos.connect.access.roles">
    <security-role name="zosConnectAccess">
      <user name="Fred"/>
    </security-role>
  </authorization-roles>

</server>
```

Figure 1: Contents of *basicSecurity.xml*

"Basic" means a user is authenticated by providing a user identity and password. In this case the user registry will be provided by the Liberty server itself. For now, it's a simple way to satisfy the security requirements of IBM z/OS Connect. In subsequent section of this exercise, SAF (e.g. RACF) will be implemented.

- ___ 31. Use the **F3** key to end the edit session.
- ___ 32. Adding this file using an *include* statement refreshes the z/OS Connect server. Enter the following MVS command to refresh the configuration **F BAQSTRT,ZCON,REFRESH**

Next, we're going to do a preliminary test of your z/OS Connect server, even though you do not yet have any services or APIs defined.

Tech-Tip: MVS commands can be entered using SDSF. In ISPF, enter the command **=SDSF.LOG** and add the / (slash) prefix to the command, e.g. **/F BAQSTRT,ZCON,REFRESH**

- ___ 33. When the message that the server configuration has been successfully updated appears in the *messages.log* file (see below), open the Firefox browser on your desktop.

```
CWPKI0803A: SSL certificate created in 5.313 seconds. SSL key file: /var/zosconnect/servers/myServer/resources/security/key.jks
CWWKS9112A: The web application security settings have changed. The following properties were modified: allowFailOverToBasicAuth=true
CWWKS9120I: Authorization roles with id="zos.connect.access.roles" have been successfully processed
CWWKG0017I: The server configuration was successfully updated in 5.481 seconds.
CWWKO0219I: TCP Channel defaultHttpEndpoint-ssl has been started and is now listening for requests on host * (IPv4) port 9443.
```

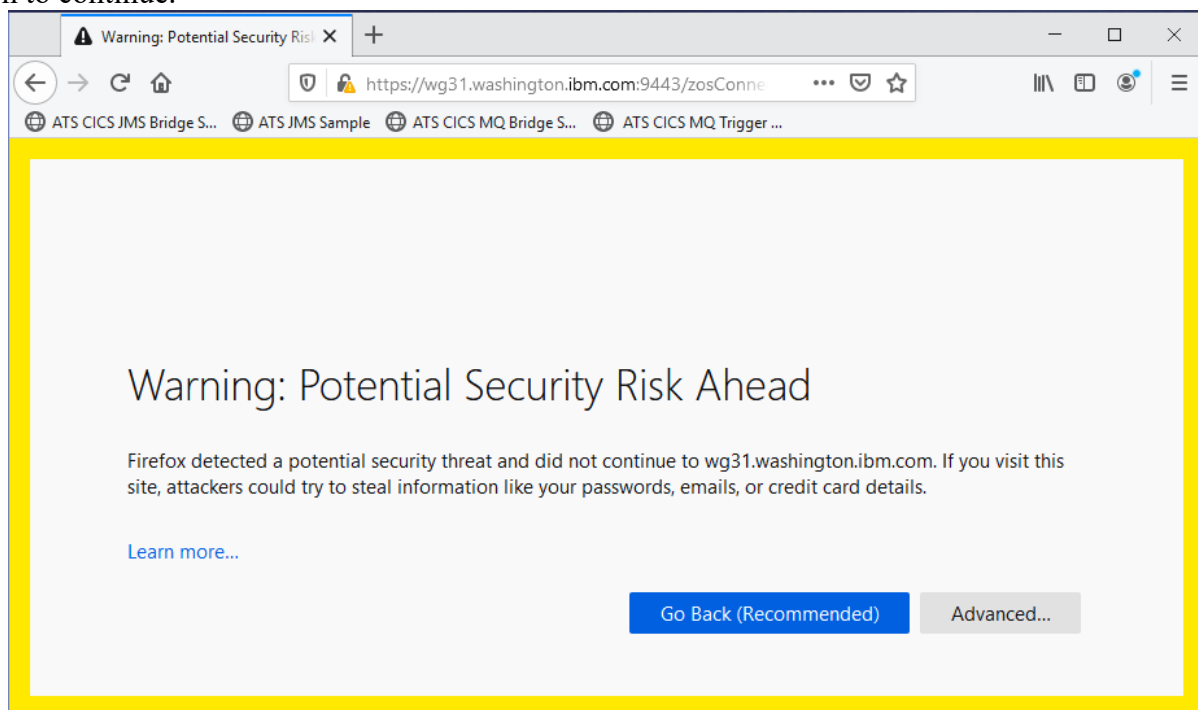

Verify the z/OS Connect server configuration

- ___ 1. Enter the following as the URL:

<https://wg31.washington.ibm.com:9443/zosConnect/apis>

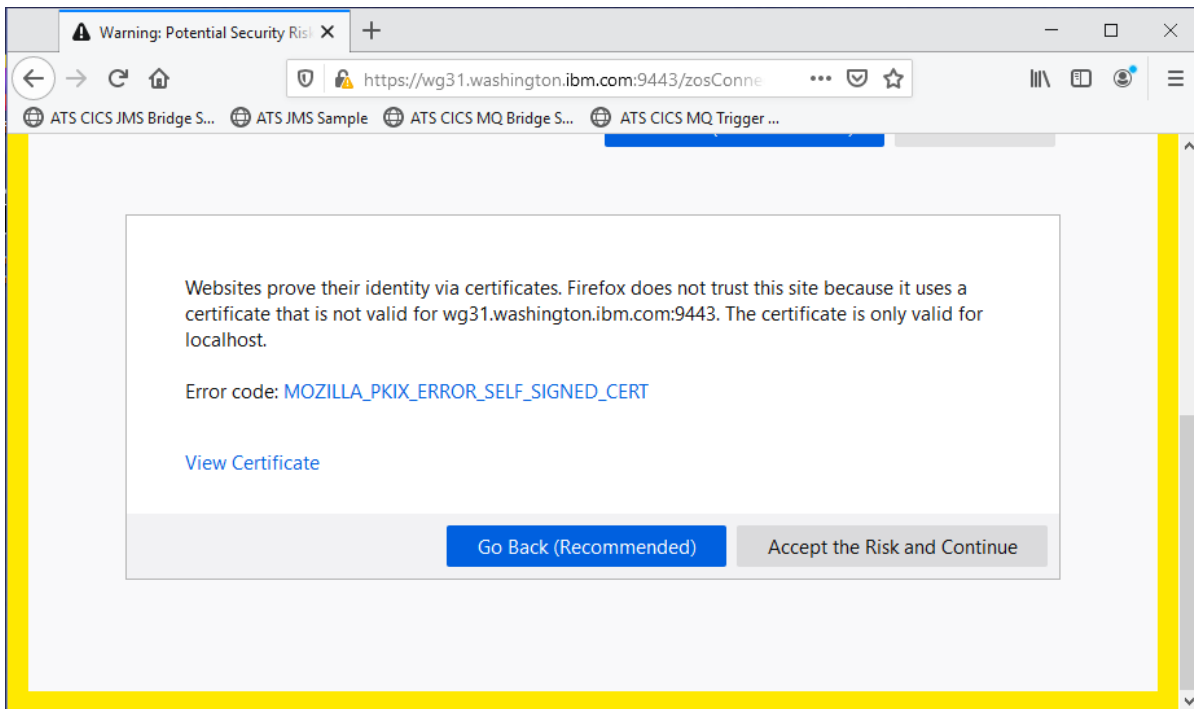
This will query the server for configured APIs. Even though you will receive a response indicating no APIs are configured, this is still a good test of connectivity to the server. But before we see the screen showing no APIs are defined, we first must address an issue where the z/OS Connect EE server is using a self-signed certificate that is not recognized by the browser, and we must authenticate to the z/OS Connect server with a user identity and password.

- ___ 2. Initially you will be challenged by Firefox because the digital certificate used by the Liberty z/OS server is self-signed (recall that we're using simple basic security for the time being). Click on the **Advanced** button to continue.



Tech-Tip: It is very important to access the z/OS Connect server from a browser prior to any testing using the Swagger UI. Accessing a z/OS Connect URL from a browser starts an SSL handshake between the browser and the server. If this handshake has not performed prior to performing any test, the test will fail with no message in the browser and no explanation. Ensuring this handshake has been performed is why you may be directed to access a z/OS Connect URL prior to using the Swagger UI during this exercise.

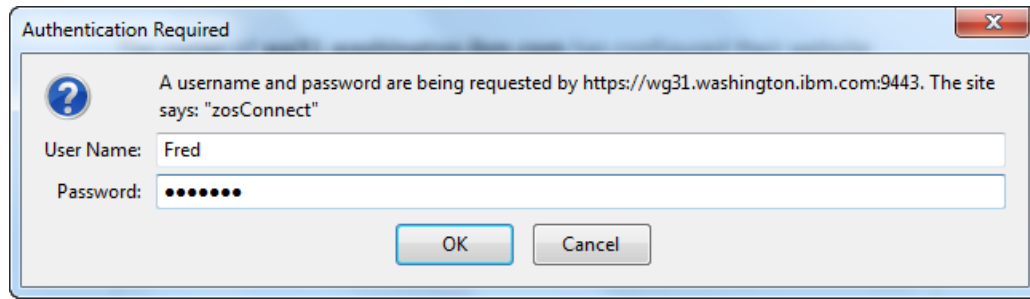
3. Scroll down and click the **Accept the Risk and Continue** button to continue.



Tech-Tip: The configuration of this Firefox browser has been configured to accept self-signed certificates and to not permanently save certificates under these circumstances. This means that these security risk screens will be displayed every time a browser is restarted until a permanent certificate is installed. This action was enabled by changing the Firefox *security.cererrors.permanentOverride* preference to *false*.

```
<basicRegistry id="basic1" realm="zosConnect">
  <user name="Fred" password="fredpwd" />
</basicRegistry>
```

___4. Next you will see a prompt you for a userid and password:

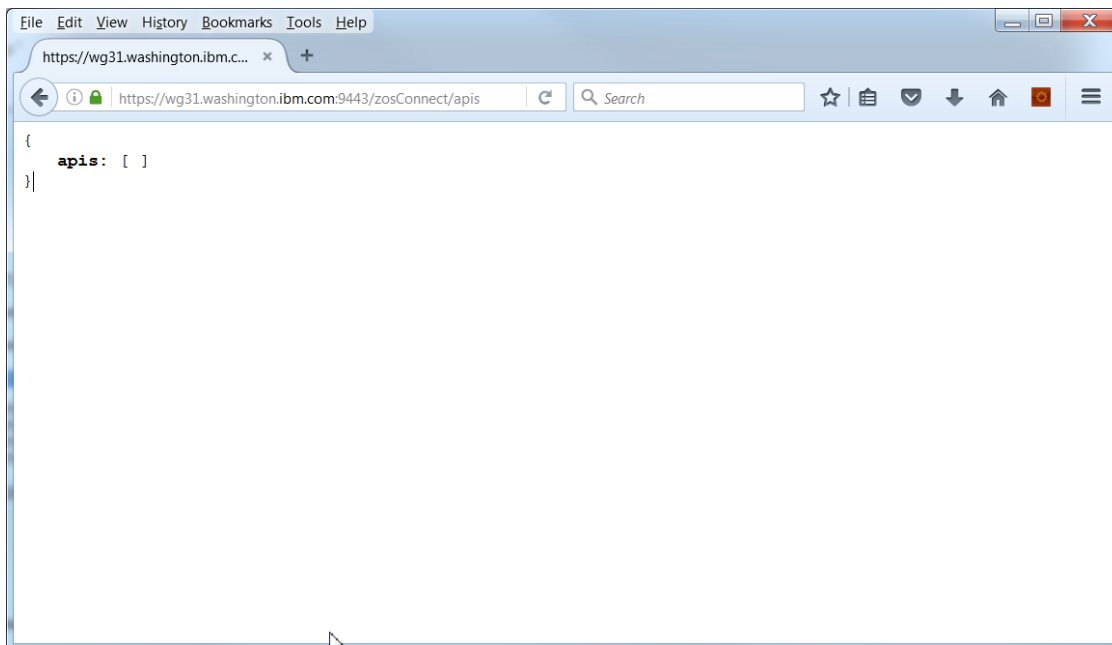


Enter a *User name* of **Fred** and a *Password* of **fredpwd** (case matters) and click **OK**.

Note: The ID and password is defined in the basicSecurity.xml file:

Our objective for security at this point in the workshop is simplicity. That's why we're using the security elements coded in the server.xml. Later we will use RACF.

___5. You should now see the following. That is the expected result since no APIs have been installed.



___ 6. Enter the following as the URL:

<https://wg31.washington.ibm.com:9443/zosConnect/services>

That will query the server for configured services. You should also get a response indicating no services are configured. This is expected.

___ 7. An alternative to using a browser, you can verify the server by using the cURL command in a *Command Prompt* session.

`curl -X GET --user Fred:fredpwd --header "Content-Type: application/json" --insecure https://wg31.washington.ibm.com:9443/zosConnect/apis`

with results:

`{"apis":[]}`

`curl -X GET --user Fred:fredpwd --header "Content-Type: application/json" --insecure https://wg31.washington.ibm.com:9443/zosConnect/services`

with results:

`{"zosConnectServices":[]}`

Summary

You customized the JCL start procedures and started both the Angel process and the server. You have added basic security for IBM z/OS Connect. Finally, you verified basic operations with the browser and optionally the cURL command.

Deploying Services and APIs

In this section, you will deploy previously generated service archive and API archives files using z/OS Connect RESTful administrative interfaces. But before the services can be deployed, the features required by the services must be installed in the running server.

Update the z/OS Connect Server Configuration

Before Services and APIs for the application can be deployed, the *server.xml* needs additional configuration information.

1. Edit the *server.xml* in */var/zosconnect/servers/myServer* and add the following *include* after the `<server description=<new server">` line.

```
<include location="${shared.config.dir}/ipic.xml"/>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<server description="new server">
<include location="${shared.config.dir}/basicSecurity.xml"/>
<include location="${shared.config.dir}/ipic.xml"
```

Tech-Tip: If a feature required by a service is not currently installed in the server, the deployment of the service will fail.

2. Including this file adds a new feature to the server, *cicsService-1.0*, which enables support for connecting to CICS regions using IP interconnectivity (IPIC). In this scenario, a CICS IPIC *TCPIPService* (see below) has been defined in the target CICS region to listen for input TCP/IP request on port 1491.

```
<server description="CICS IPIC - Catalog">

  <featureManager>
    <feature>zoscconnect:cicsService-1.0</feature>
  </featureManager>

  <zoscconnect_cicsIpicConnection id="catalog"
    host="wg31.washington.ibm.com"
    port="1491"/>

</server>
```

Figure 2: Contents of *ipic.xml*

Tech-Tip: Service names do not have to be explicitly identified in a *zosconnect_services* element. Services archive files in the *services* directory will be automatically installed either a server restart or based on the setting of the *updateTrigger* property (see *server.xml* above). One reason for adding explicit definitions for services (*zosconnect_services*) is when security attributes are required.

Note, the corresponding CICS TCPIPService definition.

```

OVERTYPE TO MODIFY                                CICS RELEASE = 0710
CEDA ALTER TCPIPService( IPIC )
TCPIPService : IPIC
GRoup : SYSPGRP
DEscription ==> DFHISAIP
Urn ==> 01491
Portnumber ==> 01491
Status ==> Open
PROtocol ==> IPic
TRAnsaction ==> CISS
Backlog ==> 00000
TSqprefix :
Host ==> ANY
(Mixed Case) ==>
Ipaddress ==> ANY
SPecifctps ==>
SCKetclose ==> No
MAXPersist ==> No
MAXDataLen ==> 000032
SYSID=CICS APPLID=CICS53Z

PF 1 HELP 2 COM 3 END          6 CRSR 7 SBH 8 SFH 9 MSG 10 SB 11 SF 12 CNCL
06/022
Connected to remote server/host wg31 using lu/pool TCP0010
  
```

3. Use the **F3** key to end the edit session.
4. Adding this file using an *include* statement refreshing the z/OS Connect server. Enter the following MVS command to refresh the configuration **FBAQSTRT,ZCON,REFRESH**

Tech-Tip: MVS commands can be only entered on any Spool Search and Display Facility (SDSF) ISPF panel if they are prefixed with a slash. To enter the MVS command **D A,L** then you must enter **/D A,L** at the Command ==> prompt on the panel. The best way to review the results of the command is to use the SDSF **LOG** command to access the system log and go to the time in the log the command was entered.

An alternative is to simply enter the SDSF **/**(slash) command at the command prompt to display the System Command Extension panel (see below).

```

Session A - wg31
File Edit View Communication Actions Window Help
Display Filter View Print Options Search Help
SDSF SYSLOG 3375.101 S0W1 S0W1 09/02/2018 2W 4,986 COLUMNS 52- 131
COMMAND INPUT ==> /
023
023 Edit Options Help
023 System Command Extension
023 ==> D A,L
023 ==>
023 Comment
023 Group Show * (F4 for list) More! +
023 ==> D A,L
023 ==> A,L
023 ==> 02D
023 ==>
023 ==>
023 ==>
023 ==>
023 ==>
023 F5=FullScr F6=Details F7=Up F8=Down F10=Save F11=Clear F12=Cancel
023 IEA6311 OPERATOR USER1 NOW INACTIVE, SYSTEM=S0W1 , LU=SC0TCP14
023 a
023 Connected to remote server/host localzos using lu
10/012
  
```

This panel provides more space for enter command parameter and also provide a means to retrieve and execute previous command. Simply place the cursor on the command in the command history and press enter. This will retrieve the command and place in the active area where a subsequent pressing of the enter key will cause the command to be executed.

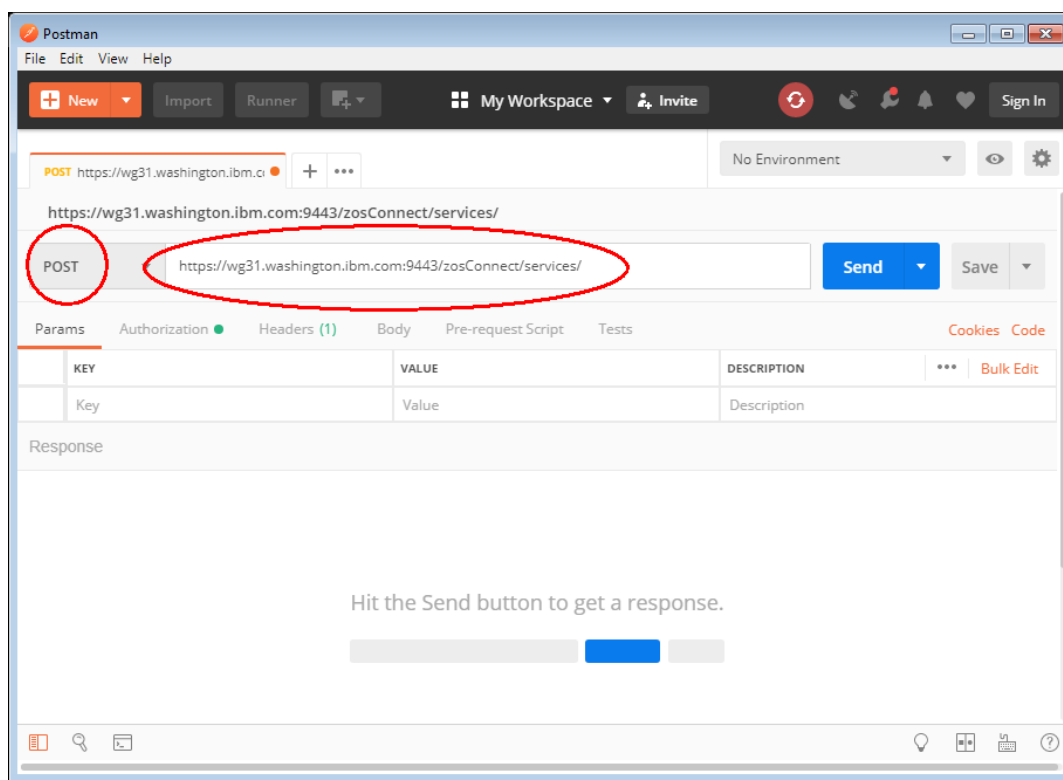
Deploy the Services

The common application artifacts installed into z/OS connect server are services and APIs (to see how these artifacts are developed, review any of the other exercises in this workshop). A *service* describes a specific interaction with a back end subsystem whereas an API describes the RESTful interface to one or more services. Both of these artifacts are archive (e.g. zipped) files. They should be installed using the z/OS Connect (OpenAPI 2) RESTful administrative interface. Two products which seem to be most popular tools for invoking z/OS Connect (OpenAPI 2) administrative RESTful APIs are *Postman* which is available for downloading from <https://www.getpostman.com/apps> and *cURL* (*client URL*) which is available for downloading from <https://curl.haxx.se/download.html>. The use of both *Postman* and *cURL* will be shown in this section of the exercise.

These instructions provide details on both. Choose either *Postman* or *cURL* to deploy the service.

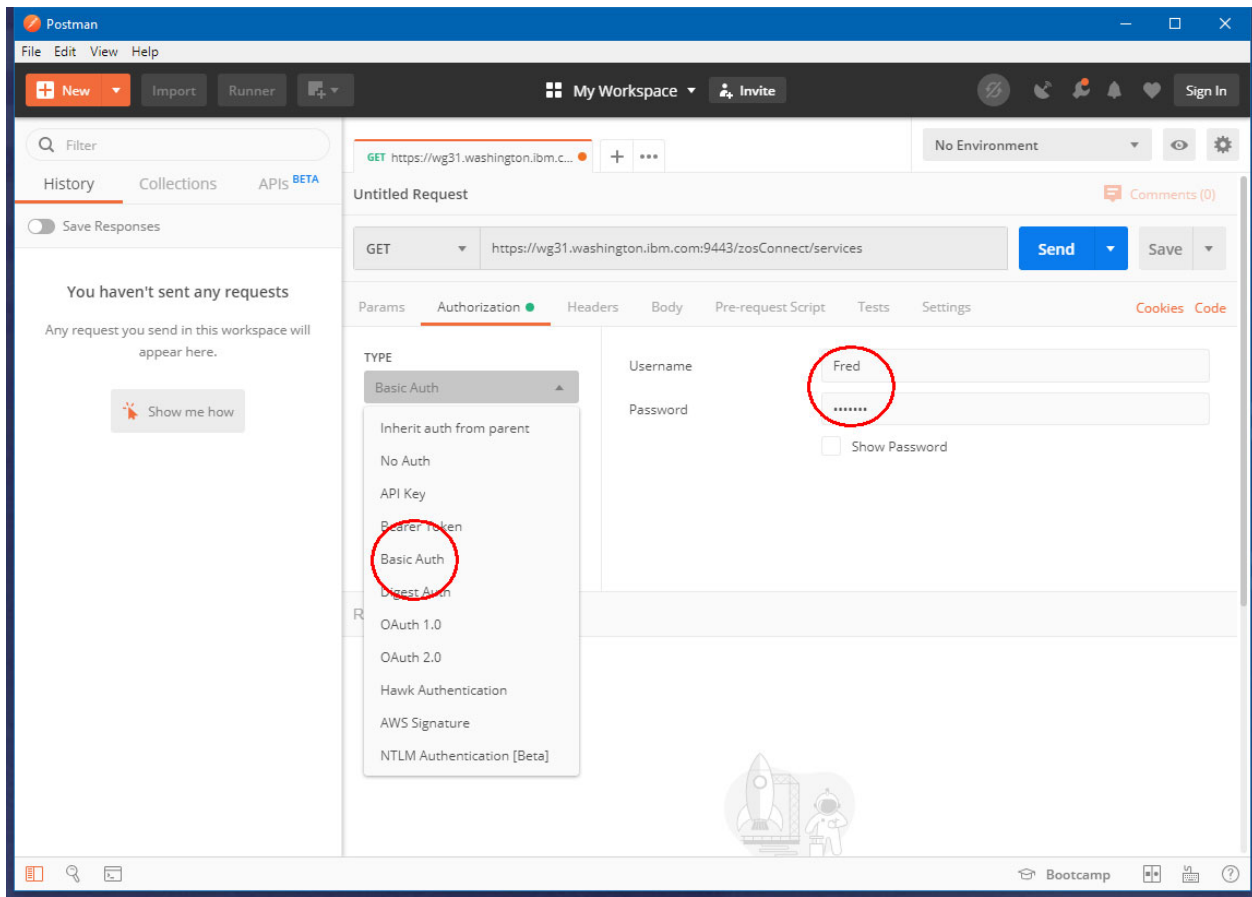
Using Postman

1. Open the *Postman* tool icon on the desktop. If necessary reply to any prompts and close any welcome messages, use the down arrow to select **POST** and enter <https://wg31.washington.ibm.com:9443/zosConnect/services/> in the URL area (see below). Do not press the **Send** button.

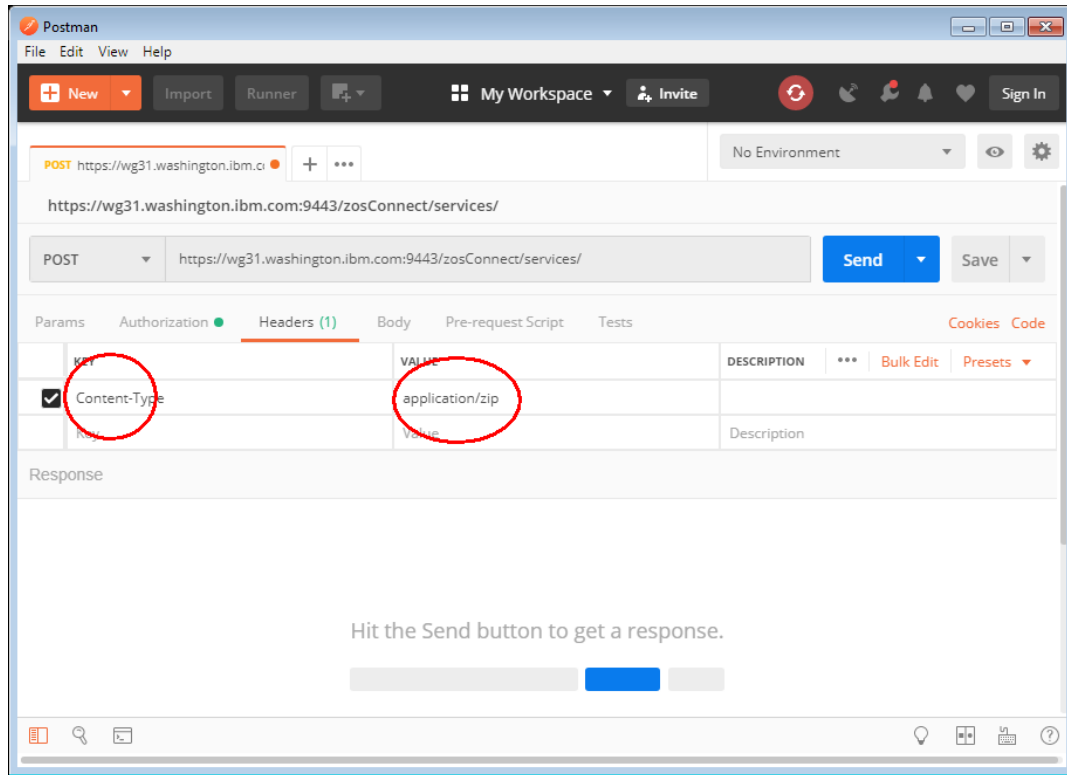


Tech-Tip: If the above Postman view is not displayed select *File* on the toolbar and then choose *New Tab* on the pull down. Alternatively, if the *Launchpad* view is displayed, click on the *Create a request* option.

2. Next, select the *Authorization* tab to enter an authorization identity and password. Use the pull down arrow to select *Basic Auth* and enter **Fred** as the username and **fredpwd** as the Password.



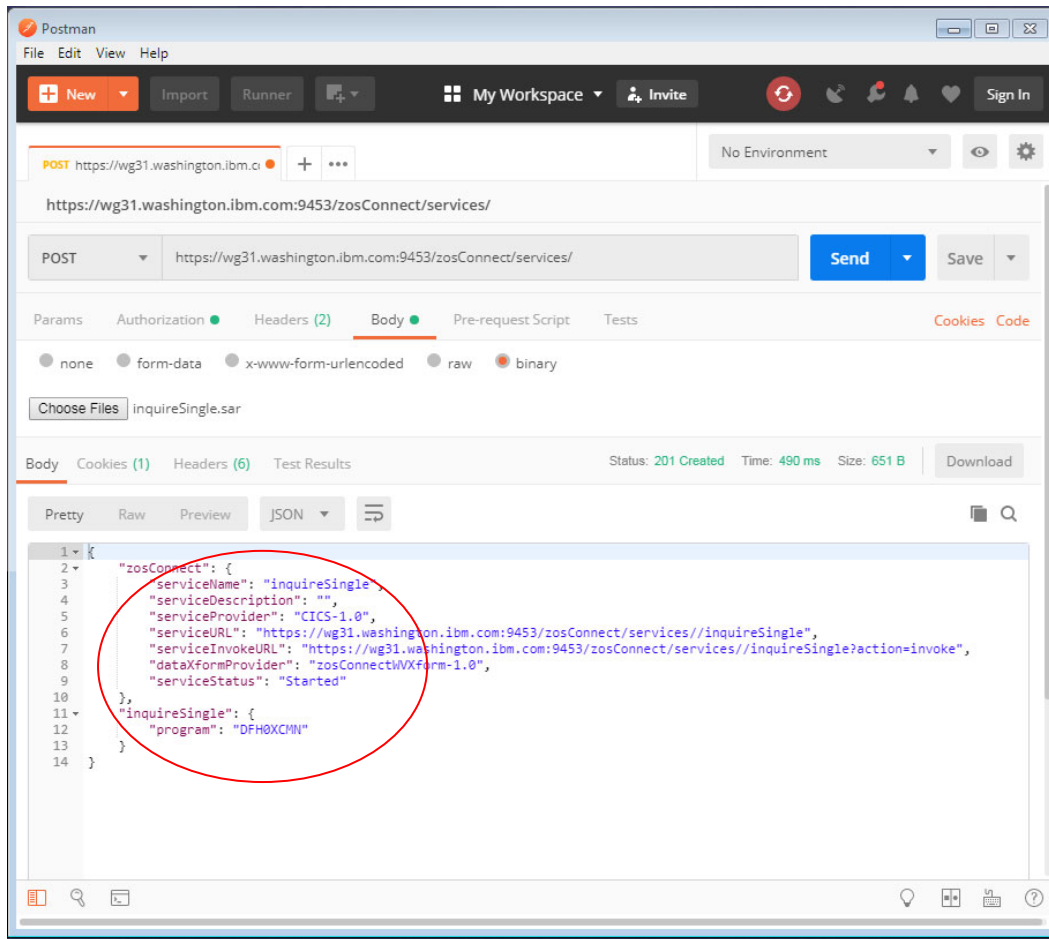
3. Next, select the *Headers* tab. Under *KEY* use the code assist feature to enter ***Content-Type***, and under *VALUE*, use the code assist feature to enter ***application/zip***.



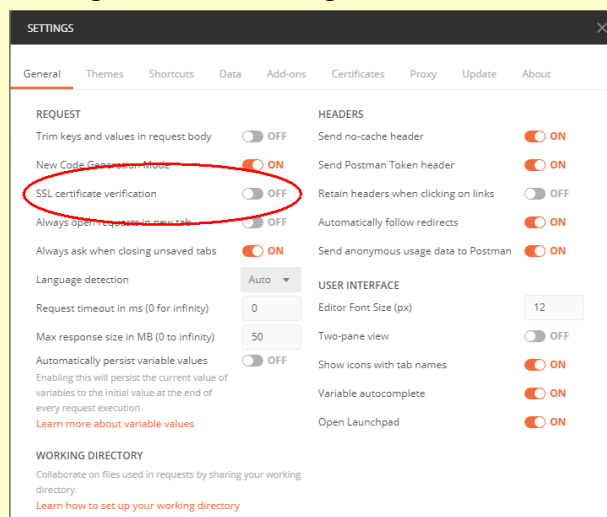
Tech-Tip: Code assist simply means that when text is entered in field, all the valid values for that field that match the typed text will be displayed. You can select the desired value for the field from the list displayed and that value will populate that field.

WebSphere Liberty on z/OS - Basic Configuration

4. Next select the *Body* tab and select the *binary* radio button. Then use the **Choose Files** button to navigate to folder *c:/z/admin*. Select the *inquireSingle.sar* file. Then press the **Send** button. A response message should come back indicating the service has been started and other details about the service.



Tech-Tip: The Postman settings have been changed to disable *SSL certificate verification*, a settings option.



Using *cURL*

The *cURL* tool provides a command line interface to REST APIs. The same administrative API service invoked with *Postman* can be invoked with *cURL* as shown here.

- ___ 1. Use the *Command Prompt* icon on the desktop to open a DOS command prompt session.
- ___ 2. In the session use the change directory (*cd*) command to go to directory *c:\z\admin*, e.g.
cd c:\z\admin
- ___ 3. Paste the command below at the command prompt and press **Enter**.

```
curl -X POST --user Fred:fredpwd --data-binary @inquireSingle.sar --header "Content-Type: application/zip" --insecure https://wg31.washington.ibm.com:9443/zosConnect/services
```

Microsoft Windows [Version 6.1.7601]

Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\workstation>***cd c:\z\admin***

```
c:\z\admin> curl -X POST --user Fred:fredpwd --data-binary @inquireSingle.sar --header "Content-Type: application/zip" --insecure https://wg31.washington.ibm.com:9443/zosConnect/services
{"zosConnect":{"serviceName":"inquireSingle","serviceDescription":"","
"serviceProvider":"CICS-
1.0","serviceURL":"https://wg31.washington.ibm.com:9443/zosConnect/services/inquireSingle",
"serviceInvokeURL":"https://wg31.washington.ibm.com:9443/zosConnect/services/inquireSingle?action
=invoke",
"dataXformProvider":"zosConnectWVXform-1.0","serviceStatus":"Started"},
"inquireSingle":{"program":"DFH0XCMN"}}}
```

Tech-Tip: In the above example:

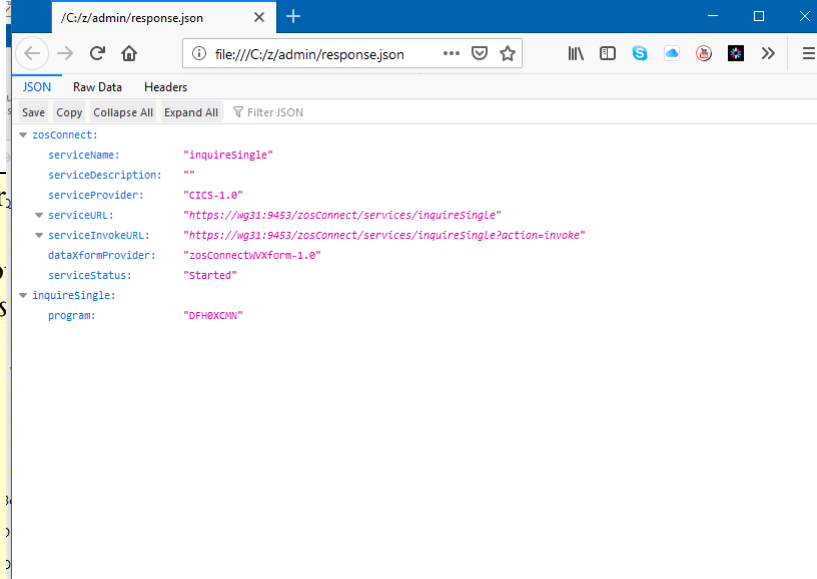
--user Fred:fredpwd could have been specified as ***--header "Authorization: Basic RnJlZDpmcmVkcHdk"***

inquireCatalog.sar is a file in the same directory in which the command is executed.

--insecure is a *cURL* directive that tells *cURL* to ignore the self-signed certificate sent by the z/OS Connect server.

The text in **green** is the JSON response message.

Tech-Tip: Another
When this directive
which then can be o
file:///c:/z/admin/res



response.json
command.firefox

4. Use either *Postman* or *cURL* to deploy the *inquireCatalog.sar* file.

```
curl -X POST --user Fred:fredpwd --data-binary @inquireCatalog.sar --header "Content-Type: application/zip" --insecure https://wg31.washington.ibm.com:9443/zosConnect/services
```

5. Use either *Postman* or *cURL* to deploy the *placeOrder.sar* file.

```
curl -X POST --user Fred:fredpwd --data-binary @placeOrder.sar --header "Content-Type: application/zip" --insecure https://wg31.washington.ibm.com:9443/zosConnect/services
```

Tech-Tip: The only differences between the commands is the name of the service archive file being deployed.

Tech-Tip: If a service needs to be redeployed, Portman or the cURL command with a PUT method can be used to stop the service, e.g., the commands are using cURL examples.

```
curl -X PUT --user USER1:USER1 --insecure https://wg31.washington.ibm.com:9483/zosConnect/services/inquireSingle?status=stopped
```

And the cURL command with a DELETE method can be used to delete the service.

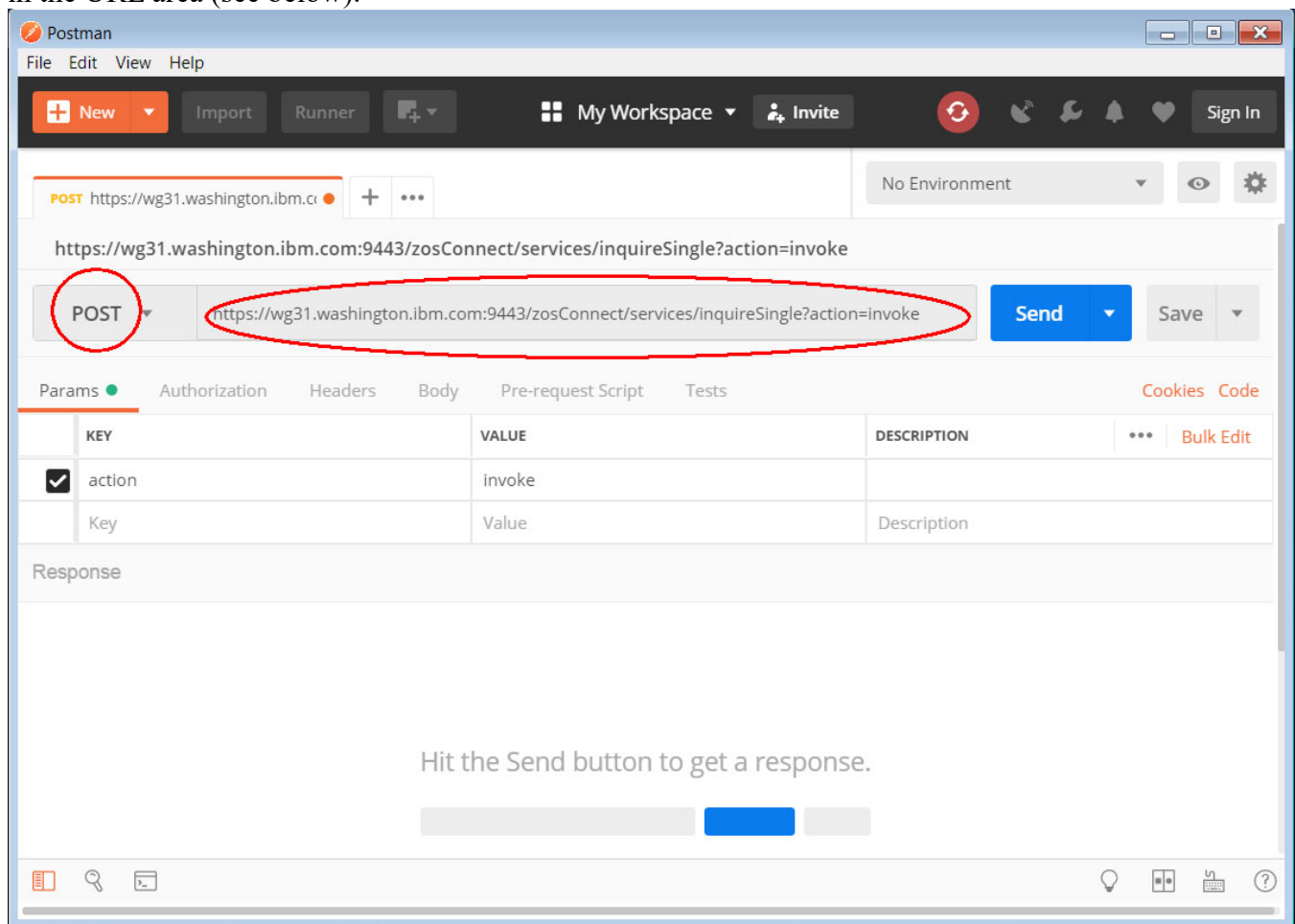
```
curl -X DELETE --user USER1:USER1 --insecure https://wg31.washington.ibm.com:9483/zosConnect/services/inquireSingle  
otherwise the redeployment of the service archive file will fail.
```

Test the Services

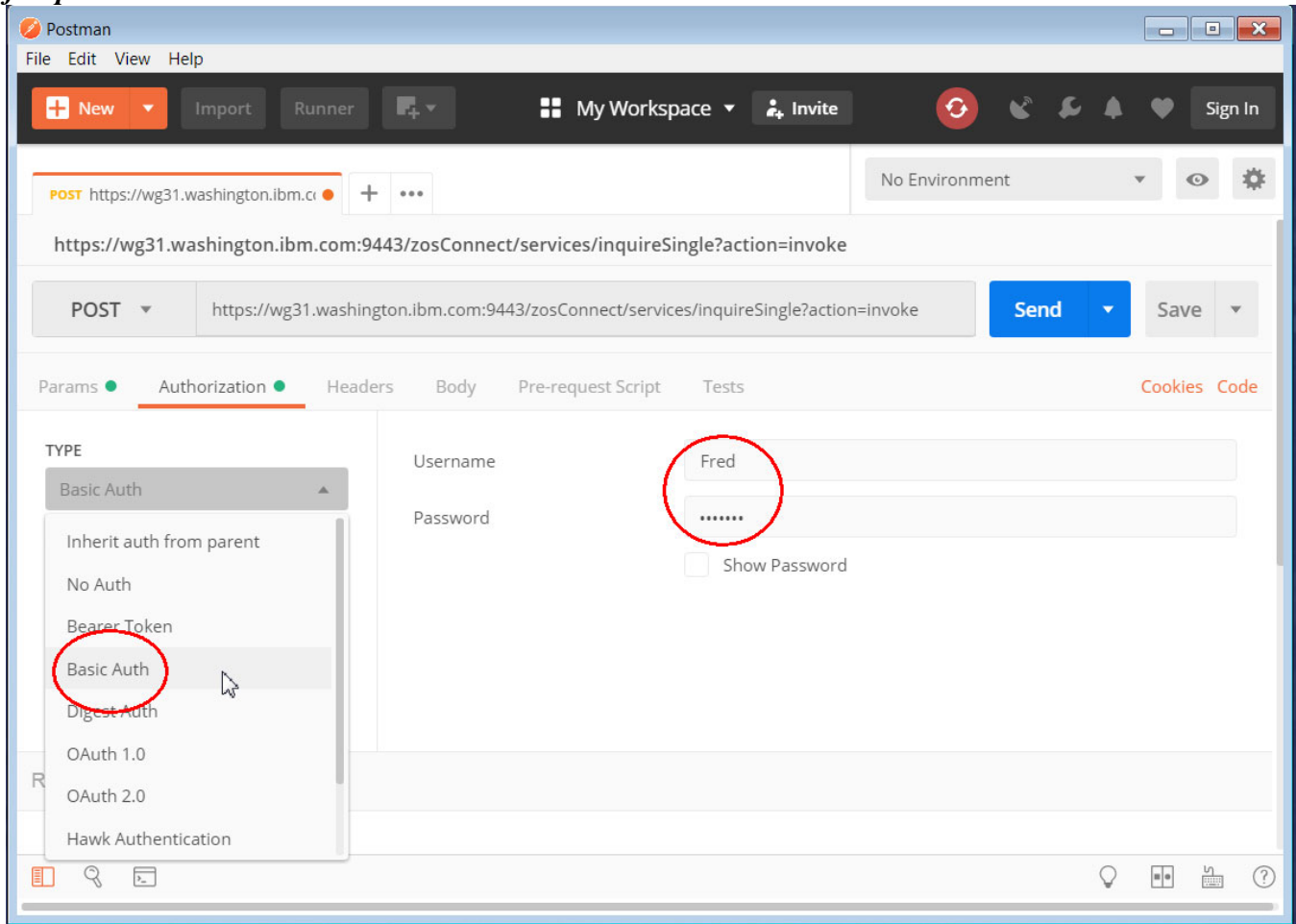
The services should be tested to ensure the infrastructure and the request and response messages are as expected.

Using Postman

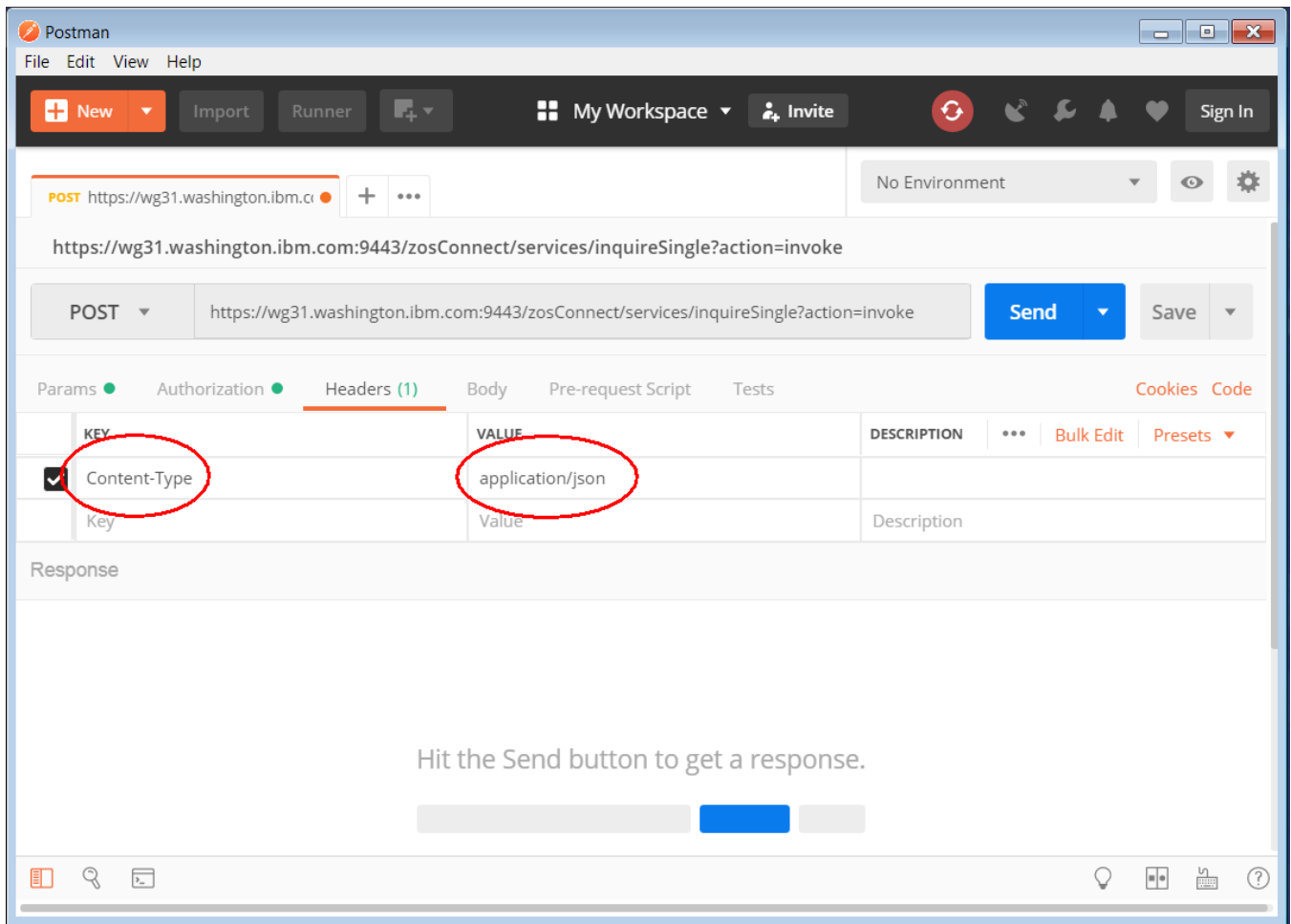
1. Open the *Postman* tool icon on the desktop and if necessary, reply to any prompts. Close any welcome messages. The use the down arrow to select **POST** and enter **<https://wg31.washington.ibm.com:9443/zosConnect/services/inquireSingle?action=invoke>** in the URL area (see below).



2. No *query* or *path* parameters are required so next select the *Authorization* tab to enter an authorization identity and password. Use the pull down arrow to select *Basic Auth* and enter ***Fred*** as the username and ***fredpwd*** as the Password.



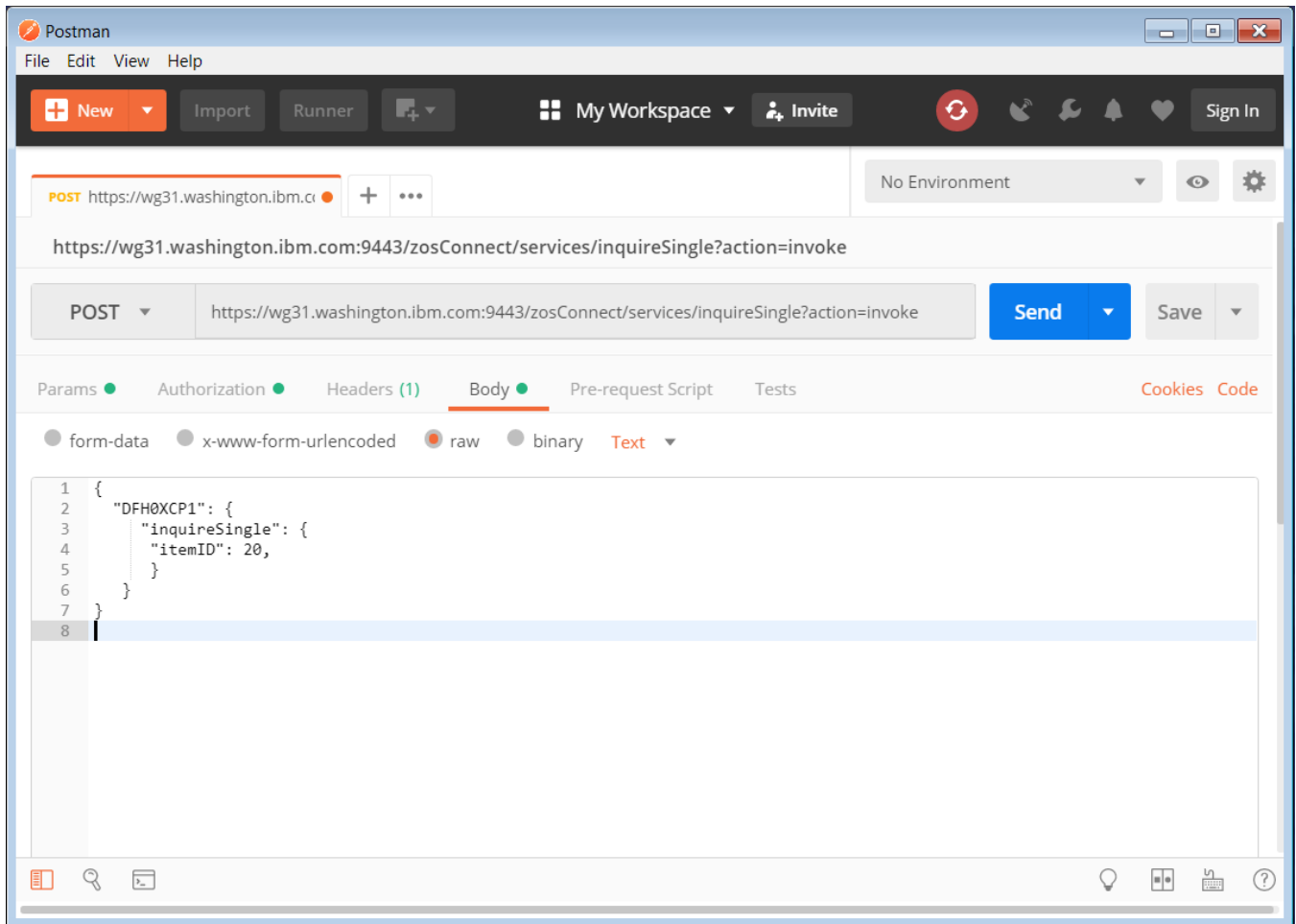
3. Next, select the *Headers* tab. Under *KEY*, use the code assist feature to enter ***Content-Type***, and under *VALUE*, use the code assist feature to enter ***application/json***.



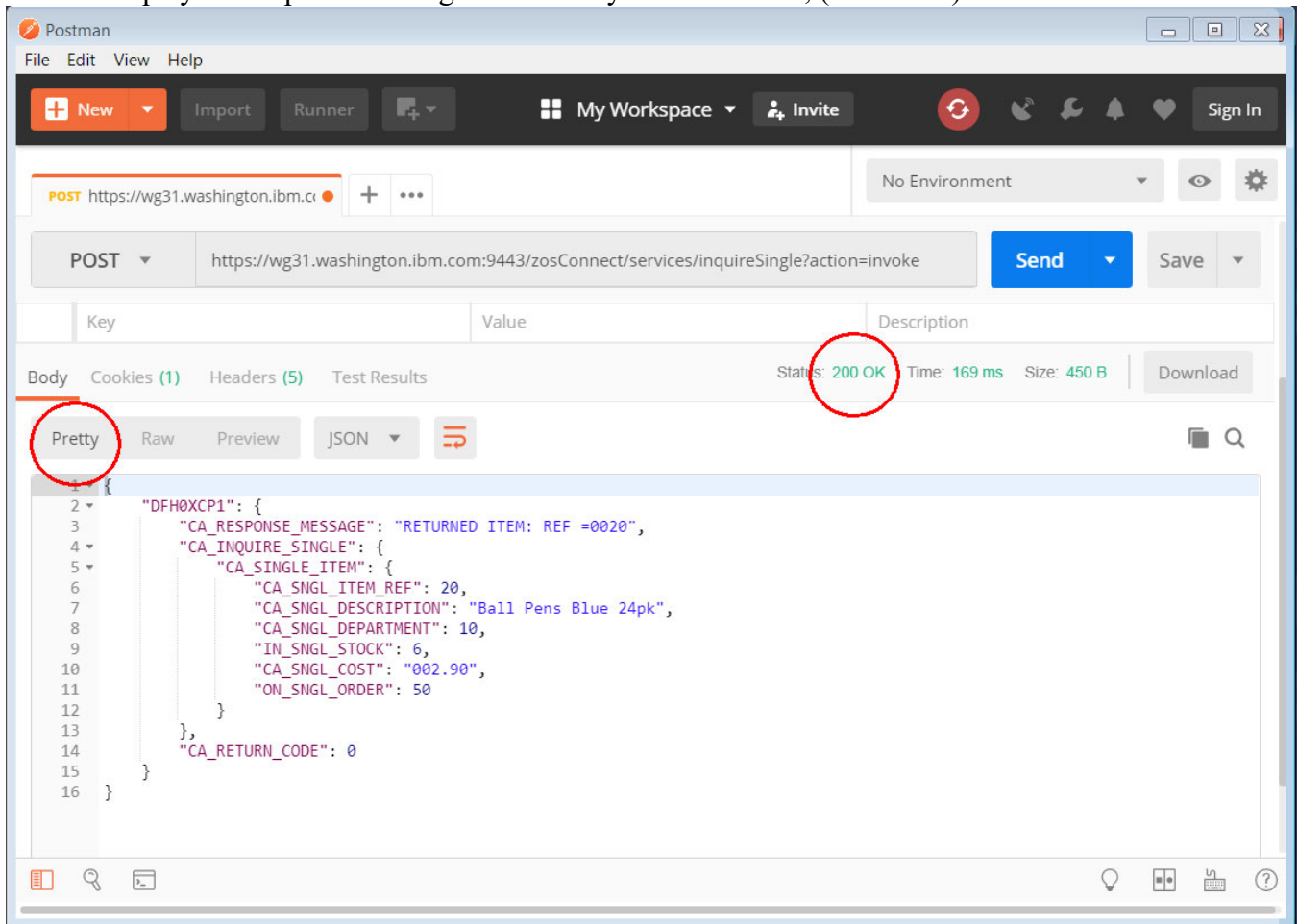
Tech-Tip: Code assist simply means that when text is entered in field, all the valid values for that field that match the typed text will be displayed. You can select the desired value for the field from the list displayed and that value will populate that field.

4. Next select the *Body* tab and select the *raw* radio button and enter the JSON message below in the *Body* area and press the **Send** button.

```
{
  "DFH0XCP1": {
    "inquireSingle": {
      "itemID": 20,
    }
  }
}
```



5. Pressing the **Send** button invokes the API. The Status of request should be 200 OK. Pressing the *Pretty* tab will display the response message into an easy to read format, (see below).



Using cURL

The *cURL* tool provides a command line interface to REST APIs. The same service just tested with *Postman* can be tested with *cURL* as shown here.

1. Use the *Command Prompt* icon on the desktop to open a DOS command prompt session.
2. In the session use the change directory (`cd`) command to go to directory `c:\z\admin`, e.g.
`cd c:\z\admin`

___3. Paste the command below at the command prompt and press **Enter**.

```
curl -X POST --user Fred:fredpwd --header "Content-Type: application/json" -d @inquireSingle.json --insecure https://wg31.washington.ibm.com:9443/zosConnect/services/inquireSingle?action=invoke
```

```
jMicrosoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\workstation>cd c:\z\admin

c:\z\admin>curl -X POST --user Fred:fredpwd --header "Content-Type: application/json"
-d @inquireSingle.json --insecure
https://wg31.washington.ibm.com:9443/zosConnect/services/inquireSingle?action=invoke
{"DFH0XCP1":{"CA_RESPONSE_MESSAGE":"RETURNED ITEM: REF
=0020","CA_INQUIRE_SINGLE":{"CA_SINGLE_ITEM":{"CA_SNGL_ITEM_REF":20,"CA_SNG
L_DESCRIPTION":"Ball Pens Blue 24pk", "CA_SNGL_DEPARTMENT":10,
"IN_SNGL_STOCK":6, "CA_SNGL_COST":"002.90", "ON_SNGL_ORDER":50}},
"CA_RETURN_CODE":0}}
```

Tech Tip: In the above example:

--user Fred:fredpwd could have been specified as **--header "Authorization: Basic RnJlZDpmcmVkcHdk"**

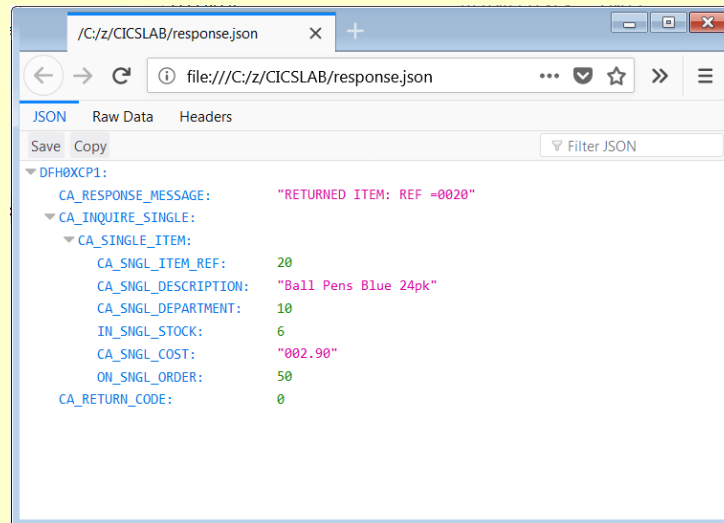
inquireSingle.json is a file in the same directory that contains the JSON request message.

--insecure is a *cURL* directive that tells *cURL* to ignore the self-signed certificate sent by the z/OS Connect EE server.

The text in **green** is the JSON response message.

Tech-Tip: Another useful cURL directive is `-o response.json`.

When this directive is used, the JSON response message is written to a file named `response.json` which then can be opened with Firefox and viewed in a more readable format, e.g. command `firefox file:///c:/z/admin/response.json`



4. The `inquireCatalog` service can be tested with *Postman* or *cURL* with URL

`https://wg31.washington.ibm.com:9443/zosConnect/services/inquireCatalog?action=invoke`

and JSON request message.

```
{
  "DFH0XCP1": {
    "inquireCatalog": {
      "startItemID": 20
    }
  }
}
```

The corresponding cURL command and results are shown below:

`curl -X POST --user Fred:fredpwd --header "Content-Type: application/json"`

`-d @inquireCatalog.json --insecure`

`https://wg31.washington.ibm.com:9443/zosConnect/services/inquireCatalog?action=invoke`

```
c:\z\admin>curl -X POST --user Fred:fredpwd --header "Content-Type: application/json"
-d @inquireCatalog.json --insecure https://wg31.washington.ibm.com:9443/
zosConnect/services/inquireCatalog?action=invoke
{"DFH0XCP1":{"CA_RESPONSE_MESSAGE":"+15 ITEMS RETURNED","CA_INQUIRE_REQUEST":{"CA_LAST_ITEM_REF":150,"CA_CAT_ITEM":[{"ON_ORDER":0,"CA_ITEM_REF":10,"CA_COST":"002.90","IN_STOCK":135,"CA_DESCRIPTION":"Ball Pens Black 24pk","CA_DEPARTMENT":10}, {"ON_ORDER":50,"CA_ITEM_REF":20,"CA_COST":"002.90","IN_STOCK":6,"CA_DESCRIPTION":"Ball Pens Blue 24pk","CA_DEPARTMENT":10}, {"ON_ORDER":0,"CA_ITEM_REF":30,"CA_COST":"002.90","IN_STOCK":106,"CA_DESCRIPTION":"Ball Pens Red 24pk","CA_DEPARTMENT":10}, {"ON_ORDER":0,"CA_ITEM_REF":40,"CA_COST":"002.90","IN_STOCK":80,"CA_DESCRIPTION":"Ball Pens Green 24pk","CA_DEPARTMENT":10}, {"ON_ORDER":0,"CA_ITEM_REF":50,"CA_COST":"001.78","IN_STOCK":83,"CA_DESCRIPTION":"Pencil with eraser 12pk","CA_DEPARTMENT":10}, {"ON_ORDER":40,"CA_ITEM_REF":60,"CA_COST":"003.89","IN_STOCK":13,"CA_DESCRIPTION":"Highlighters Assorted 5pk","CA_DEPARTMENT":10}, {"ON_ORDER":20,"CA_ITEM_REF":70,"CA_COST":"007.44","IN_STOCK":101,"CA_DESCRIPTION":"Laser Paper 28-lb 108 Bright 500Vream","CA_DEPARTMENT":10}, {"ON_ORDER":0,"CA_ITEM_REF":80,"CA_COST":"033.54","IN_STOCK":25,"CA_DESCRIPTION":"Laser Paper 28-lb 108 Brig
```

The *placeOrder* service can be tested using *Postman* or *cURL* with URL:

https://wg31.washington.ibm.com:9443/zosConnect/services/placeOrder?action=invoke

and JSON request message.

```
{
  "DFH0XCP1": {
    "orderRequest": {
      "itemID": 70,
      "orderQuantity": 1
    }
  }
}
```

The corresponding cURL command and results are shown below:

curl -X POST --user Fred:fredpwd --header "Content-Type: application/json" -d @placeOrder.json --insecure https://wg31.washington.ibm.com:9443/zosConnect/services/placeOrder?action=invoke

```
c:\z\admin>curl -X POST --user Fred:fredpwd --header "Content-Type: application/json"
-d @placeOrder.json --insecure https://wg31.washington.ibm.com:9443/zosC
onnect/services/placeOrder?action=invoke
{"DFH0XCP1":{"CA_RESPONSE_MESSAGE":"ORDER SUCCESSFULLY PLACED","CA_RETURN_CODE":0}}
```

Deploy and Test the API

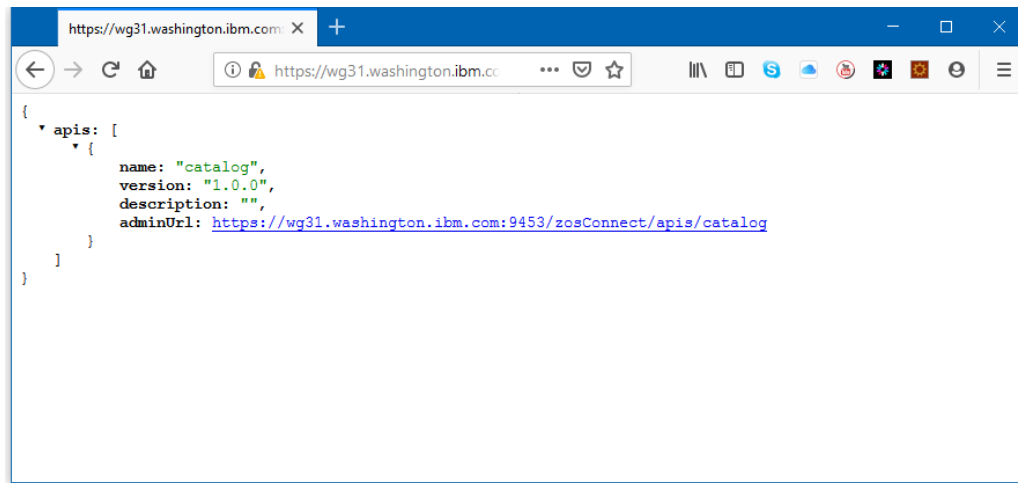
The API artifact should be deployed using z/OS Connect RESTful administrative interface. cURL will be used in this section, but Postman could have been used also.

___1. Deploy the API archive file using cURL. Paste the command below at the command prompt and press **Enter**.

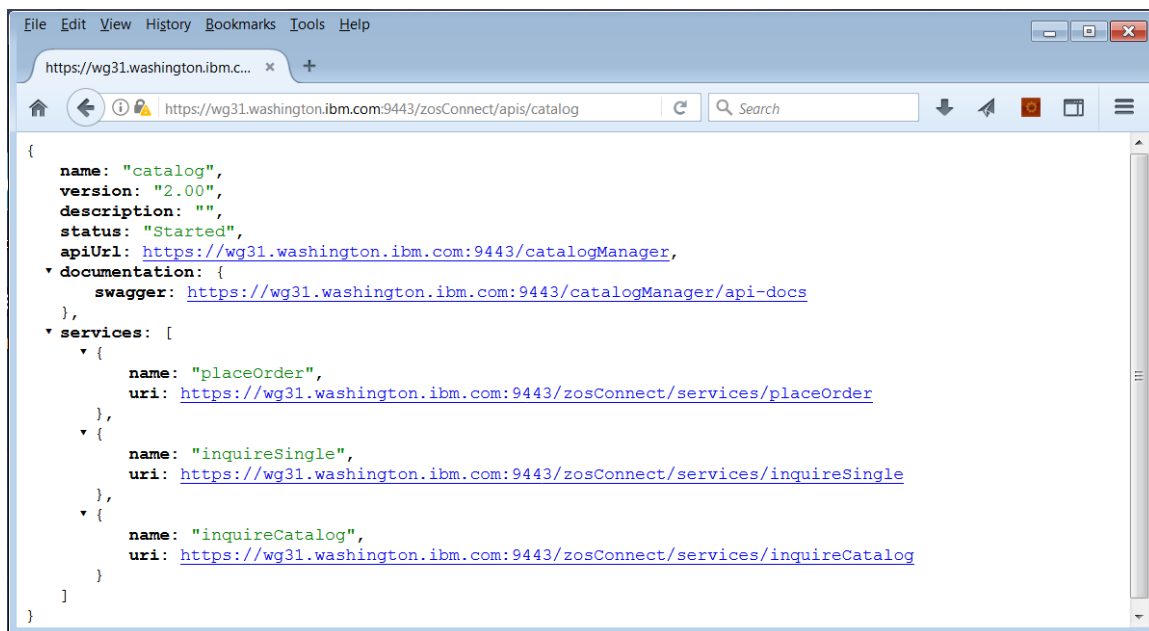
```
curl -X POST --user Fred:fredpwd --data-binary @catalog.aar --header "Content-Type: application/zip" --insecure https://wg31.washington.ibm.com:9443/zosConnect/apis
```

```
C:\z\admin>curl -X POST --user Fred:fredpwd --data-binary @catalog.aar --header
"Content-Type: application/zip" --insecure
https://wg31.washington.ibm.com:9443/zosConnect/apis
{"name":"catalog","version":"1.0.0","description":"","status":"Started",
"apiUrl":"https://wg31.washington.ibm.com:9453/catalog",
"documentation":{"swagger":"https://wg31.washington.ibm.com:9443/catalog/api-docs"},
"services":[
{"name":"placeOrder","uri":"https://wg31.washington.ibm.com:9443/zosConnect/services/place
Order"},
{"name":"inquireSingle","uri":"https://wg31.washington.ibm.com:9443/zosConnect/services/inq
uireSingle"},
{"name":"inquireCatalog","uri":"https://wg31.washington.ibm.com:9443/zosConnect/services/in
quireCatalog"}
]}
```

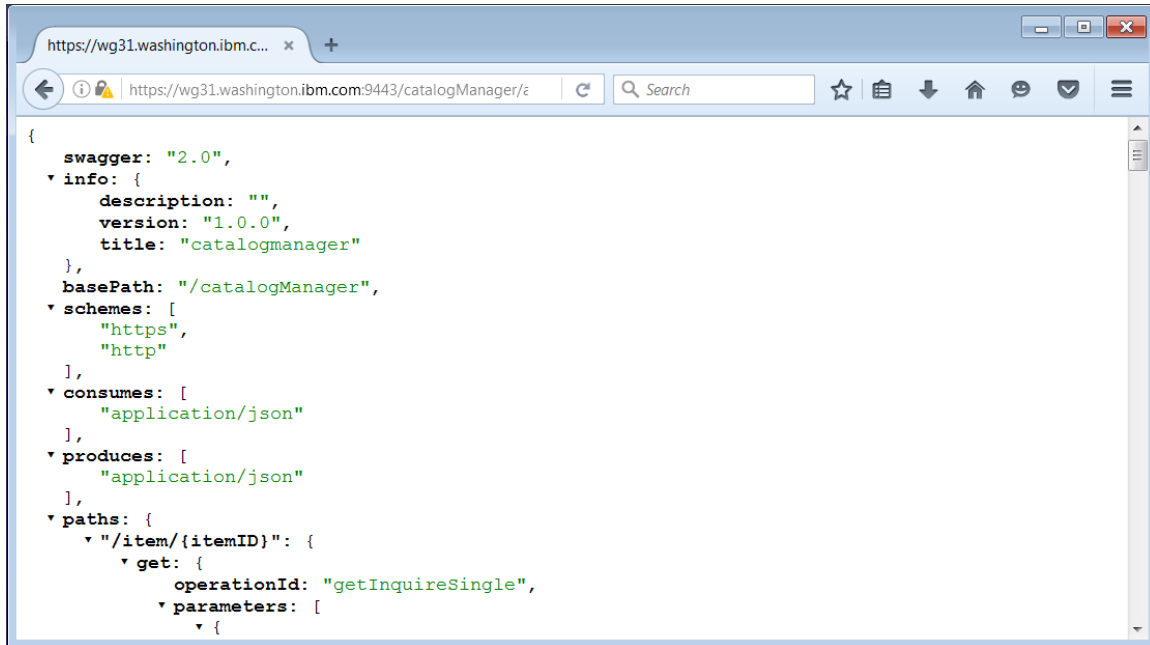
2. Next enter URL <https://wg31.washington.ibm.com:9443/zosConnect/apis> in the Firefox browser and you should see the window below. The *catalog* API now shows as being available.



3. If you click on *adminUrl* URL, the window below should be displayed.

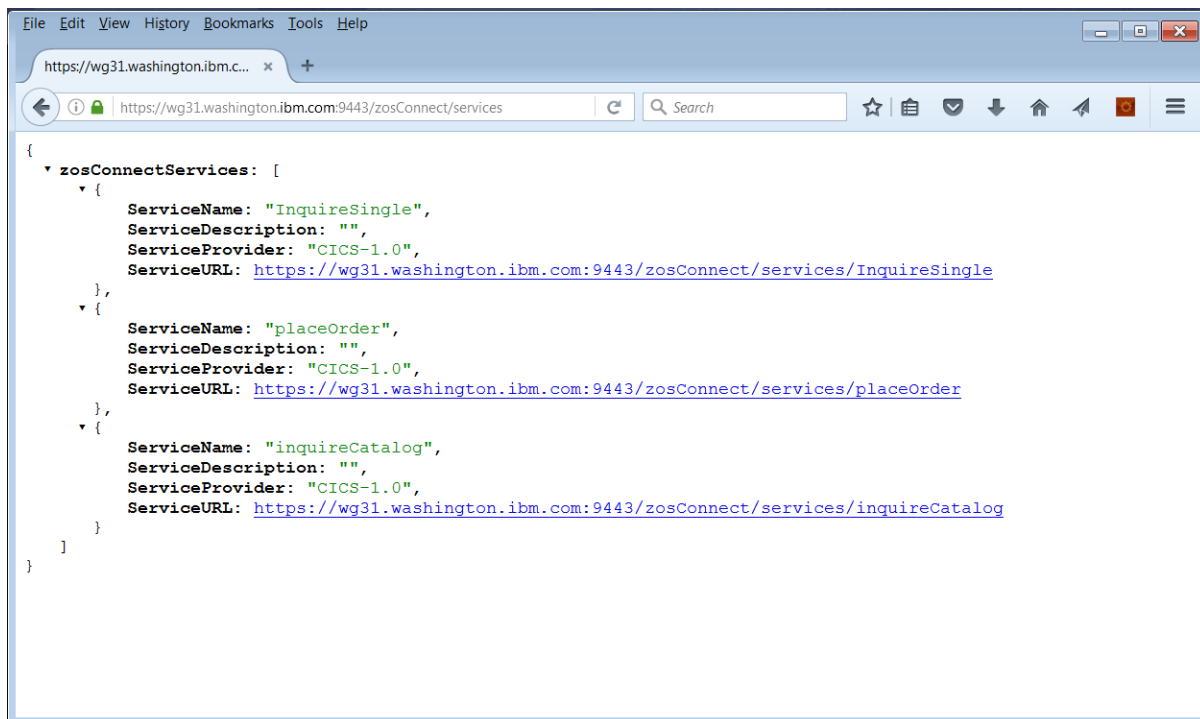


4. Next click on the *swagger* URL and you should see the Swagger document associated with this API:



Explore this Swagger document and you will see the results of the request and response mapping performed earlier. This Swagger document can be used by a developer or other tooling to develop REST clients for this specific API.

5. Next enter URL <https://wg31.washington.ibm.com:9443/zosConnect/services> in the Firefox browser and you should see the window below. The three services are now available.



Use cURL or Postman to test the API.

```
curl -X GET --user Fred:fredpwd --insecure  
https://wg31.washington.ibm.com:9443/catalog/item/0010
```

```
curl -X GET --user Fred:fredpwd --insecure  
https://wg31.washington.ibm.com:9443/catalog/items?startItemID=0010
```

```
curl -X POST --user Fred:fredpwd --data @placeOrder.json --header "Content-Type:  
application/json" --insecure https://wg31.washington.ibm.com:9443/catalog/order
```

The available catalog items are listed below.

Item#	Description	Dept	Cost	In Stock	On Order
0010	Ball Pens Black 24pk	010	002.90	0135	000
0020	Ball Pens Blue 24pk	010	002.90	0006	050
0030	Ball Pens Red 24pk	010	002.90	0106	000
0040	Ball Pens Green 24pk	010	002.90	0080	000
0050	Pencil with eraser 12pk	010	001.78	0083	000
0060	Highlighters Assorted 5pk	010	003.89	0013	040
0070	Laser Paper 28-lb 108 Bright 500/ream	010	007.44	0102	020
0080	Laser Paper 28-lb 108 Bright 2500/case	010	033.54	0025	000
0090	Blue Laser Paper 20lb 500/ream	010	005.35	0022	000
0100	Green Laser Paper 20lb 500/ream	010	005.35	0003	020
0110	IBM Network Printer 24 - Toner cart	010	169.56	0012	000
0120	Standard Diary: Week to view 8 1/4x5 3/4	010	025.99	0007	000
0130	Wall Planner: Eraseable 36x24	010	018.85	0003	000
0140	70 Sheet Hard Back wire bound notepad	010	005.89	0084	000
0150	Sticky Notes 3x3 Assorted Colors 5pk	010	005.35	0036	045
0160	Sticky Notes 3x3 Assorted Colors 10pk	010	009.75	0067	030
0170	Sticky Notes 3x6 Assorted Colors 5pk	010	007.55	0064	030
0180	Highlighters Yellow 5pk	010	003.49	0088	010
0190	Highlighters Blue 5pk	010	003.49	0076	020
0200	12 inch clear rule 5pk	010	002.12	0014	010
0210	Clear sticky tape 5pk	010	004.27	0073	000

Optional

If you are familiar with CICS Execution Diagnostic Facility (EDF), start a 3270-terminal session with CICS. Clear the screen and enter CICS transaction **CEDX CSMI**. When you repeat any of the above test, you should be able to trace the flow of the request through CICS.

```

Session A
File Edit View Communication Actions Window Help
TRANSACTION: CSMI PROGRAM: DFHMIRS TASK: 0000092 APPLID: CICS53Z DISPLAY: 00
STATUS: PROGRAM INITIATION

EIBTIME      = 184802
EIBDATE      = 0117226
EIBTRNID     = 'CSMI'
EIBTASKN     = 92
EIBTRMID     = '/AC3'

EIBCPOSN     = 0
EIBCALEN     = 0
EIBRID       = X'00' AT X'1530011A'
EIBFN        = X'0000' AT X'1530011B'
EIBRCODE     = X'000000000000' AT X'1530011D'
EIBDS        = .....
+ EIBREQID   = .....

ENTER: CONTINUE
PF1 : UNDEFINED PF2 : SWITCH HEX/CHAR PF3 : END EDF SESSION
PF4 : SUPPRESS DISPLAYS PF5 : WORKING STORAGE PF6 : USER DISPLAY
PF7 : SCROLL BACK PF8 : SCROLL FORWARD PF9 : STOP CONDITIONS
PF10: PREVIOUS DISPLAY PF11: EIB DISPLAY PF12: UNDEFINED

MA A 01/001
Connected to remote server/host wg31a using lu/pool TCP001

```

Summary

You have verified the API. The API layer operates above the service layer you defined. The API layer provides a further level of abstraction, allows a more flexible use of HTTP verbs, and provides better mapping of data via the API editor function.